

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI-590018, KARNATAKA



**Project Report
on**

**A FLEXIBLE FPGA APPROACH TO IMAGE
CRYPTOGRAPHY**

Submitted in partial fulfillment of the requirements for the VII semester

Project Work Phase-II

Bachelor of Engineering

in

Electronics and Communication Engineering

by

**LAGINENI SREENITHAI (1CD22EC082)
MULUKURI VALLESH NATH (1CD22EC178)
RAGHAVENDRA SM (1CD22EC133)
CHANDRAKIRAN SM (1CD22EC034)**

Under the Guidance of

Prof. CHAITHRA S

Assistant Professor

Department of Electronics and Communication Engineering



**Department of Electronics and Communication Engineering
CAMBRIDGE INSTITUTE OF TECHNOLOGY, BANGALORE - 560 036**

2025 – 2026

CAMBRIDGE INSTITUTE OF TECHNOLOGY

K.R. Puram, Bangalore – 560036

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

It is Certified that **Mr. L Sreenithai, Mr. M Vallesh Nath, Mr. Raghavendra SM and Mr. Chandrakiran SM** bearing USN: 1CD22EC082, 1CD22EC178, 1CD22EC133 and 1CD22EC034 respectively, are bonafide students of **Cambridge Institute of Technology, Bangalore** and have completed requirements of the project entitled “**A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY**” in partial fulfillment of the requirements for VII semester **Bachelor of Engineering in Electronics and Communication Engineering** of **Visvesvaraya Technological University, Belagavi** during academic year 2025 – 2026. It is certified that all Corrections/Suggestions indicated for Internal Assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of **project work Phase – II(BEC786)** prescribed for the Bachelor of Engineering degree.

Prof. Chaithra S
Project Guide
Dept. of ECE, CITech

Dr. Kavitha M V
Project Co-Ordinator
Dept. of ECE. CITech

Dr. Nagesh K N
Vice Principal & Head of the Dept.
Dept. of ECE. CITech

Dr. B V Ravishankar
Principal
CITech

External Viva
Name of the Examiners

Signature with date

- 1.
- 2.

DECLARATION

We, **Mr. L Sreenithai, Mr. M Vallesh Nath, Mr. Raghavendra SM and Mr. Chandrakiran SM** bearing **USN 1CD22EC082, 1CD22EC178, 1CD22EC133 and 1CD22EC034** respectively, are students of VII semester, Electronics and Communication Engineering, Cambridge Institute of Technology, hereby declare that the project entitled “**A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY**” has been carried out by us and submitted in partial fulfillment of the course requirements **Project work Phase – II** of VII semester **Bachelor of Engineering in Electronics and Communication Engineering** as prescribed by **Visvesvaraya Technological University, Belagavi**, during the academic year 2025 – 2026.

We also declare that, to the best of our knowledge and belief, the work reported here does not form part of any other report on the basis of which a degree or award was conferred on an earlier occasion on this by any other student.

Date:

Place: Bengaluru

L Sreenithai(1CD22EC082)

M Vallesh Nath(1CD22EC178)

Raghavendra SM(1CD22EC133)

Chandrakiran SM(1CD22EC034)

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to Shri **D. K. Mohan, Chairman**, Cambridge Group of Institutions, Bangalore, India for providing excellent Infrastructure and Academic Environment at CITECH without which this work would not have been possible.

I extremely thankful to **Dr. B. V Ravishankar, Principal**, Cambridge Institute of Technology, Bengaluru for providing me Academic ambience and Laboratory facilities to in and everlasting motivation to carry out this work and shaping our careers.

I express my sincere gratitude to **Dr. Nagesh K.N, Vice Principal & HOD**, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for his stimulating guidance, continuous encouragement and motivation throughout the course of present work.

I also wish to extend my thanks to our **Project Co-Ordinator Dr. Kavitha M V , Associate Professor**, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for their unstilted support, valuable guidance and help throughout the seminar related work.

I also wish to extend my thanks to our **Project Guide Prof. Chaithra S, Assistant Professor**, Department of Electronics and Communication Engineering, Cambridge Institute of Technology, Bengaluru for her critical, insightful comments, guidance and constructive suggestion and support to improve the quality of this seminar work.

I would like to thank all the faculty members and non-teaching staff of Dept. of ECE, for their constant support. And I would like to thank our parents and friends for their constant moral and financial support.

L Sreenithai(1CD22EC082)

M Vallesh Nath(1CD22EC178)

Raghavendra SM(1CD22EC133)

Chandrakiran SM(1CD22EC034)

ABSTRACT

In the rapidly evolving landscape of digital media, ensuring the confidentiality and integrity of image transmission has become a critical priority, particularly within medical diagnostics, industrial surveillance, and IoT networks. Traditional software-based encryption methods often face significant bottlenecks when run on General-Purpose Processors (GPPs), failing to meet the strict latency and throughput requirements of modern real-time systems. To address these limitations, this project presents a robust Hardware/Software Co-design cryptography scheme implemented on the PYNQ-Z2 development board. By leveraging the Xilinx Vivado High-Level Synthesis (HLS) toolchain, computationally intensive encryption algorithms are transformed from C++ into optimized Register Transfer Level (RTL) logic. This allows the system to offload block-based image processing directly onto the Programmable Logic (PL), utilizing the FPGA's massive parallelism to bypass the sequential processing overhead of standard CPUs. A high-bandwidth AXI Direct Memory Access (DMA) architecture is employed to establish a seamless data pipeline, ensuring that image data flows continuously between the processing system and the hardware accelerator without stalling.

Complementing this high-performance hardware is a user-friendly software layer designed to democratize FPGA usage. A Python-based interface running on the Processing System (PS) handles image acquisition, pre-processing, and visualization, effectively bridging the gap between complex hardware logic and accessible software control. Experimental validation of this architecture confirms a paradigm shift in performance, achieving sub-second encryption and decryption latency and a substantial boost in throughput compared to software-only approaches, all while maintaining minimal power consumption. Beyond immediate performance gains, the system's modular design ensures scalability and extensibility, making it "future-ready" for integration with AI-powered diagnostic pipelines. This makes the proposed method an ideal solution for secure, high-speed image transmission in resource-constrained, latency-sensitive edge computing environments.

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
	CERTIFICATE	i
	DECLARATION	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	INTRODUCTION	1-6
2	LITERATURE SURVEY	7-12
3	DESIGN AND IMPLEMENTATION	13-24
4	FLOWCHART	25-34
5	RESULTS & DISCUSSION	35-39
6	APPLICATIONS	40-41
7	CONCLUSION	42
8	FUTURE SCOPE	43-44
	REFERENCES	45-46
	Appendix - I (Procedural Steps)	47-49
	Appendix – II (Plagiarism and AI Reports)	
	Appendix – III (Copy of Published Paper)	
	Appendix – IV (Submission in IEEE Journal)	

LIST OF FIGURES

Sl. No.	List of Figures	Page Number
3.1	Block diagram of Proposed Methodology	13
3.2	PYNQ Z2 FPGA DEVELOPMENT BOARD	18
3.3	ZYNQ Clock Distribution Architecture	20
4.1	Development Process of Image Cryptography on PYNQ-Z2	28
4.2	AES-128 Flowchart	30
5.1	Input Image	35
5.2	Encrypted Image Using AES IP	35
5.3	Decrypted Image Using AES IP	35
5.4	Design Timing Summary	36
5.5	Power Analysis of AES IP from VIVADO	37

LIST OF ABBREVIATIONS

Sl.No.	List	Description
1	AES	Advanced Encryption Standard
2	RSA	Rivest–Shamir–Adleman (cryptosystem)
3	CPU	Central Processing Unit
4	IoT	Internet of Things
5	FPGA	Field-Programmable Gate Array
6	PS	Processing System
7	PL	Programmable Logic
8	HLS	High-Level Synthesis
9	DMA	Direct Memory Access
10	SoC	System on Chip
11	RTL	Register Transfer Level
12	HDL	Hardware Description Language
13	AXI	Advanced eXtensible Interface
14	DDR	Double Data Rate (SDRAM)
15	RAM	Random Access Memory
16	JTAG	Joint Test Action Group (interface)
17	USB	Universal Serial Bus

18	UART	Universal Asynchronous Receiver-Transmitter
19	DSP	Digital Signal Processor
20	HDMI	High-Definition Multimedia Interface
21	PHY	Physical Layer (transceiver)
22	RGMII	Reduced Gigabit Media Independent Interface
23	SPN	Substitution-Permutation Network
24	WNS	Worst Negative Slack
25	WHS	Worst Hold Slack
26	WPWS	Worst Pulse Width Slack
27	TNS	Total Negative Slack
28	THS	Total Hold Slack
29	MB/s	Megabytes per second
30	Mbps	Megabits per second
31	II	Initiation Interval
32	ML	Machine Learning
33	I/O	Input/Output
34	EHR	Electronic Health Record
35	PACS	Picture Archiving and Communication Systems
36	HIPAA	Health Insurance Portability and Accountability Act
37	SWaP	Size, Weight, and Power

CHAPTER 1

INTRODUCTION

The proliferation of the Internet of Things (IoT) has fundamentally shifted the computational paradigm from centralized cloud infrastructures to edge computing. Cloud-based Processing of ML Models (Cloud Processing) is transitioning to an Embedded System Model (Embedded at Source). The transition is taking place because of the requirement for low latency and energy efficient solutions. It recently became possible to perform sophisticated classification inference of ML Models directly on Field Programmable Gate Arrays (FPGAs) with PYNQ-Z2. By performing ML model inference on an FPGA, the computer can perform computations with much faster speeds and consume much less energy than when utilizing a cloud server to perform the same computation by using an architecture such as ResNet.[1]. With the increasing complexity of heterogeneous IoT environments, an ever-growing number of diverse devices generate vast quantities of data, leading to substantial vulnerability in multiple areas. Therefore, capturing image data from an edge device and transmitting it across networks poses security risks. Therefore, it is critical to implement strong cryptographic standards (such as AES-128) adequately and securely [2].

As the number and complexity of heterogeneous IoT environments continue to rapidly grow and change, a larger and more varied selection of devices will generate ever increasing amounts of data in a manner that creates uncertainty or increased risk in several areas of traditional operations. Consequently, the process of capturing image data from an IoT "edge" device and transmitting it through a wired or wireless network creates inherent risks with respect to the security of that data. Thus, it is imperative that an organization ensures the implementation and use of high quality cryptographic standard(s) (such as the AES-128 standard) in a thorough and complete manner. [3]. Regarding quality of data; although quality of the data used is of the utmost importance during this process. VHDL hardware-based image enhancement provides superior performance over traditional DSP through the use of FPGAs, whereby parallel processing and increased efficiency allow for the filtering and adjustment of brightness to occur in a much more efficient manner than traditional software-based methods [4].

Encryption has been utilized throughout history in different forms and have become more creative with time. One such way of using scrambling or encrypting an image is by utilizing a

"Chaotic" scrolling format to scramble an image's contents (i.e., to change the order of the pixels). This form of encoding allows for the movement of pixels from one location in a two-dimensional matrix to another location in a two-dimensional matrix without changing the underlying number format of the pixel data contained in the matrix. The result is that although the spatial locations change, the same numerical values continue to represent the same pixel data. This results in greater levels of distortion than when the underlying pixel data representation is left unchanged even if the spatial coordinates were scrambled [5]. A Two-Factor authentication system that combines Least Significant Bit (LSB) Steganography with Symmetric Cryptography will create a much more complex method of authenticating users. The complexity of the method helps to provide a secure method for sending Hello world communications via the web with integrity [6].

The application of these hardware-accelerated schemes extends into critical domains such as healthcare. In biomedical monitoring, deep convolutional neural networks (DCNNs) deployed on edge devices have achieved high accuracy in classifying electrocardiogram (ECG) signals, demonstrating that complex deep learning models can operate effectively on boards like the PYNQ-Z2 with minimal parameters [7]. Furthermore, novel encryption methodologies, such as those inspired by DNA subsequence operations combined with logistic chaotic maps, provide expansive key spaces and resistance to statistical attacks, offering alternative avenues for securing sensitive medical imagery [8].

Despite these advancements, a significant gap remains in unifying efficient processing with rapid development cycles. Hardware/software co-design methodologies are essential to bridge this gap, allowing for the creation of real-time embedded solutions - such as robot localization systems - that rival PC-based performance while reducing development time [9]. In the example above, multiple techniques were utilized in an integrated fashion that combined various components of both YOLOv4-tiny detection & ZUC to secure the transfer of detections with a single PLD programmable logic device. [10].

The above results support establishing a dedicated niche in automating medical diagnosis concerning hardware acceleration. Conventional methods of applying software alone to develop tools for accurately and rapidly diagnosing infectious TB or pneumonia from CXR digital imaging do not typically provide sufficient accuracy and significant speed of processing for diagnoses in the medical environment, but recent studies indicated that low-power, light-weight (DCNN) based systems developed with a cloud CPU (using an HLS to convert those systems to

FPGA) provide, on average, an accuracy of approximately 84%. The varying speed of how expedient a diagnostic can be made between the above example systems likely results from the fixed-point data types that are used within the PYNQ-Z2 system architecture, ultimately resulting in far less time spent to produce the final inference while retaining the same level of accuracy compared to any other commercially available general-purpose processing system.[11].

Edge Computing, which is often used by Resource-Constrained Edge Devices, Rarely Utilizes Both Hardware Acceleration And Cryptographic Security In The Same Application. Although Both Hardware Acceleration And Cryptographic Security Are Critical To Providing A Secure Solution For Edge Devices, They Are Typically Implemented Separately. The Current State Of The Art For Secure Edge Devices Is Either Focused On The Complexity Of The Encryption Or On Maximizing The Speed Of The Inference Process. The Integrated Framework That We Present Joins The Use Of FPGA Technology With The Light Weight Deep Convolutional Neural Network For Medical Diagnostics, And The High Throughput Module For Chaotic Encryption. Our Integrated Framework Is Developed Using Hardware/Software Co-Design Methods; It Leverages The Full Capabilities Of The PYNQ-Z2 Platform To Conduct Clinical Level Analyses Of Highly Sensitive Physiological Data In Real Time While Protecting The End-To-End Integrity Of The Physiological Data Through The Use Of Extremely Low Latency. The remainder of this paper details the system architecture, implementation methodology, and experimental performance results.

1.1 Problem Statement

The old encryption from software, like AES and RSA, is good for text but not good for images. Images with a great deal of data when utilizing large image files due to the pixel similarities are essentially large (>2 MB) files therefore requiring a lot of CPU resources to encrypt using conventional software. This restricts the ability to send images rapidly such as through email or live video from CCTV cameras as well as causing problems with devices having limited power sources (e.g., IoT sensor nodes).

Currently, there is no fast, low-power solution for securely encrypting images. For this reason, there is an urgent requirement for solutions that use hardware.

Encryption using conventional techniques when dealing with video, excellent compression techniques will typically use nearly 90% of the CPU for encryption and this causes delays in processing and increases latency in transmitting files. This is a significant performance bottleneck in applications where real time results are critical, for instance, when video feeds coming from UAV's, immediate and timely processing must occur; encryption delays will

ultimately result in what is known as, stuttering, which makes the video useless while waiting for encryption.

The rapid increase of IoT and mobile sensor device utilization has also resulted in very strict hardware limitations placed on these devices. Due to this fact, these battery-driven devices are mostly not able to provide sufficient processing to maintain intensive software-based encryption without rapid energy drain. Consequently, hardware acceleration is needed instead of purely software-based solutions to overcome these challenges.

1.2 Scope of the Project

The purpose of this project is to develop a hardware solution for encrypting and decrypting images utilizing the PYNQ-Z2 hardware platform. In this endeavor, a C++ implementation of the LED Cipher will be created, converted to an application implementation in hardware using Vivado HLS and built into a software interface developed in Python for use with image processing of the encrypted and unencrypted files and control of the overall system. The connection between the hardware and software components will occur using an AXI-DMA interface allowing for high speed transfer of data between the Processing System (PS) and Programmable Logic (PL). The system will focus on processing grayscale images by dividing these images into blocks prior to encrypting. Testing will be performed on the PYNQ-Z2 to verify the system's operation and performance. No new ciphers will be developed or alternative boards utilized.

1.3 Objectives of the Project

- To make the LED block cipher in C++ and get it ready for Vivado HLS, so it can go on the PYNQ-Z2 FPGA.
- To use Vivado HLS to turn the C++ code into a hardware IP core and put it on the board.
- To write Python software that can take a image, cut it into blocks, send data to the FPGA, and get the encrypted image back.
- To check that the system works good, meaning a image encrypted and then decrypted looks exactly the same as the original.
- To test the system and see how much faster it is than software, and if it uses less power.

1.4 Motivation for the Project

We chose this Suction Cup feature for several reasons, the most important being the increased need to secure medical images in today's world. If we allow a medical image to be taken, or possibly altered, the patient could suffer far-reaching consequences. Many times we had

observed how slowly Software Encryption encrypts images written to smaller devices; therefore, we determined that we needed to find alternatives. We were very interested in learning about hardware design as well as developing physical products using FPGA (Field Programmable Gate Arrays) technology so we decided to investigate ways in which we could use Software Coding Tools (i.e. Vivado HLS) to generate hardware from code; this concept was too interesting for our team to pass up. Through this project, we will create an actual functional solution while also expanding our knowledge of Hardware & Security.

1.5 Proposed Methodology

The way we will build the system is step by step, using the PYNQ-Z2.

1. Create Algorithm and Hardware: First, we developed the LED cipher in C++, verifying that it functions properly. Then we imported this C++ code into Vivado HLS, which converts the code into a hardware description (RTL). We utilized various options within HLS to create faster and smaller hardware circuitry.
2. Assemble System: We imported the IP core created in HLS into Vivado. Inside Vivado, we created a Block Design to route the IP Core to the ARM Processor via AXI interconnects. In addition, we added an AXI DMA to allow quick movement of data from RAM to the hardware. Finally, we used Vivado to generate the Bitstream File that contains the configuration needed for the FPGA.
3. Develop Software: We developed a Python application for the PYNQ board. This program accesses the API provided by PYNQ to control the FPGA. The program will accept an image as input and convert it to bytes before slicing the byte string into blocks ("slices") for the implementation of the LED cipher, using the DMA to transfer the blocks to the FPGA and retrieve the processed blocks before reconstructing the original image from the processed blocks.
4. The Process of Operating the System. Once we have completed the setup for the system, the first thing will happen in our Python program is to load the appropriate configuration (or bitstream) into the FPGA to create our encryption hardware. Once this is complete, when we receive image block data, the Python program sends that block's pixel information to memory, makes a request for the hardware through Direct Memory Access (DMA), then makes a request for the hardware to encrypt, and waits for the encrypted image block to be returned. However, in the case of decryption, the same process will occur; we will just go through the instructions in reverse. By using this method, any time that the FPGA hardware is available, we can utilize it

quickly, and the entire workflow will appear to be completed in near real-time.

This project explores using FPGA (Field-Programmable Gate Arrays) for the encryption of data instead of traditional software-based encryption methods. FPGAs offer an alternative to GPPs (General Purpose Processors) because they allow users to custom-configure electronics, using the FPGA device as a medium to include those configurations. This makes FPGAs much faster and more efficient than CPUs when performing most types of image processing, due to this "massive parallelism" that allows for the simultaneous processing of many different data blocks.

The chosen development board is the PYNQ-Z2 and it contains a dual-core ARM Cortex-A9 processor and a programmable logic (PL) that creates a heterogeneous computing environment. This project will illustrate how the hardware/software co-design environment is used to create fast real-time image encryption applications. Less resource-intensive application code will run on the ARM processor, while the particularly compute-intensive image encryption algorithms will run on the programmable logic (PL) within the FPGA to eliminate potential performance issues created by the ARM processor.

CHAPTER 2

LITERATURE SURVEY

R. Jaimon et al. (2025) This study developed an edge-based machine-learning (ML) solution that used image classification models deployed on the PYNQ-Z2 platform. The work was based on a TensorComputeUnit (TCU) architecture to improve performance and resource efficiency over software implementations. It was demonstrated that offloading to the TCU significantly improved performance versus the software-only implementations. It provides an implementation methodology for utilizing complex deep learning models in embedded environments with limited computational resources[1].

K. Lata and S. Saini (2020)By developing an AES-128 algorithm through forced hardware-software co-simulation, the authors successfully addressed some of the security issues faced by Internet of Things (IoT) systems; the authors found that dividing the work of encrypting data among both processors and custom hardware elements provides a good compromise between extreme performance and flexibility. The results from the co-simulation support the ability of the architecture to permit safe transmission of photographs while retaining high-speed transmission characteristics over other wireless mediums. This research supports a cooperative design methodology that addresses the security issues of all connected embedded devices.[2].

E. H. Y. Wajih et al. (2023)The research presents a newvariant of a Chaotic Symmetric Key Stream Cipher developed for encryption of images transmitted in relation to the Internet of Things (IoT). Stream of random generated cryptotext coming from the mathematical functions expressing chaos (chaotic Maps) cannot be performed via statistical methods to analyze them. This research provides evaluations of performance based on Security Strength, computational load, and implementation in Low-powered devices using IoT Communication technology and ultimately proves that chaos-based encryption methods can solve today's limitations caused by the increasing number of resource Constraints on the IoT Security. [3].

C. S. et al. (2023) The research involved using MATLAB and Verilog as tool sets to develop algorithms for enhancing images and comparing hardware and software implementations of those algorithms, including an analysis of hardware vs software implementations in terms of how much resource (i.e., time, memory, etc.) is consumed by each type of implementation and

what the associated latency is. The results of this research show that while MATLAB allowed for quicker prototyping of enhancement algorithms, the hardware implementation of enhancement algorithms based on Verilog resulted in a significantly reduced amount of latency for real-time situations. Hence, this research exemplifies how important it is to weigh the advantages and disadvantages when defining a "development environment" for digital image processing systems [4].

A. J. Mansor et al. (2018) FPGA technology has been used to create an Image Scrambling algorithm based upon a Chaotic System. The algorithm effectively obscures adjacent pixel correlation by rearranging pixel locations because of the sensitivity of chaotic maps in relation to their initial conditions. Successful implementation of the algorithm on an FPGAs shows the speed of execution and supports Real-Time, Secure Image Communications. The results of this study also demonstrate that using FPGA Technology allows for faster implementation of very complex Cryptographic Algorithms for Encrypted Digital Content. [5].

P. Ghosh et al. (2024) Created a two-level authentication mechanism which utilizes both Steganography and Cryptography as the primary means to secure Image and Text Data. The system first encrypts the information, then it embeds this encrypted information in an image file, thereby increasing the level of security through imperceptibility. By using a conjunction of hardware implementation methods and twofold authentication/verification, it has been demonstrated that this combination of techniques reduces unauthorized access while preserving the integrity of the data. This work contributes to multimodal security. The concept of combining different forms of security to enhance security in Embedded Systems is demonstrated [6].

S. Tiwari and P. R. Muduli (2022) In the present work, we demonstrate a method of utilizing a convolution neural network (CNN) along with a field programmable gate array (FPGA) such as the PYNQ-Z2 board to provide efficient classification of ECG signals obtained from an electrocardiogram. This paper demonstrates how to implement programmer logic for optimising those layers of the CNN that are computationally heavy, thereby demonstrating efficient classification of multiple types of cardiac arrhythmias with an excellent degree of accuracy. This paper demonstrates how the PYNQ-Z2 board represents a very flexible and powerful platform on which to deploy AI-based health care solutions within an edge computing environment, and illustrates the increasing use of FPGA-SoC platforms in portable biomedical diagnostics [7].

Q. Zhang et al. (2012) The limitations of current encryption techniques for images were examined with the development of a new image encryption algorithm using DNA subsequences as the primary basis. By combining the advantages of DNA computing's vast parallelism with biological processes of elongation, truncation, and so forth, images can have their pixel values mixed up effectively. A series of statistical tests and security analyses demonstrated that this proposed algorithm has a very large keyspace and is very sensitive to changes—the combination of these two traits results in strong protection from brute-force attacks. The work represents the first use of biological computing in the creation of secure digital images [8].

S. Pedre et al. (2012) A way to develop hardware and software together has been created for embedded imaging systems in real time. Task profiling before execution helps to separate the functions of the regular CPU and the custom hardware accelerator while ensuring maximum throughput. By examining the time required for completing multiple tasks, it was shown that the co-design of these components significantly outperformed time-constrained solutions that only employed software. This research provides a starting point for optimizing embedded system architectures to perform multimedia processing [9].

D. Liu et al. (2025) We have created a Target Detection architecture using Field Programmable Gate Array (FPGA) technology and Secure Transmission of Encrypted Data from the Target Detection system through to an Edge-Based Intelligence System. Classic Edge-Based Intelligence systems have associated Embedded Systems to leverage several functionalities on FPGA systems. Researchers are investigating a Target Detection algorithm using the full potential of an FPGA (with multiple parallel processors) to provide near real-time operations for processing detected targets. The Target Detection Architecture is coupled with the hardware accelerated Encryption Architecture for the secure transmission of both Target and Video Feed data feeds. This paper presents a holistic methodology for creating Embedded Systems solutions capable of delivering data securely, with Lower Latency Times and enabling practical Real-Time Operations [10].

S. R. Dwibedi and P. K. Biswal (2024) This research created an effective embedded system that predicts if someone has had a brain stroke using Cost-Sensitive Machine Learning (CSML) with the PYNQ-Z2 board. The study focused on finding the best combination of accuracy vs power requirements: by using the PYNQ platform to run the computational requirements for the Machine Learning (ML) model at the edge, the system allows the user to offload the ML

computational elements to a remote location with minimal latency. This research also demonstrated that high levels of predictive accuracy appropriate for medical-based services can be obtained by using the AUC-ROC and F1 Score metrics. Thus, this research demonstrates the potential of leveraging edge-computing platforms for critical, latency-constrained applications in healthcare [11].

V. Kumar et al. (2024) Using a Wavelet-based Adaptive Boosting method, we created a cuffless BP system on the PYNQ-Z2. The PPG feature extraction was accomplished using Wavelets, while the machine learning algorithms used for acceleration were run on an FPGA. Our results indicate that accurate BP estimation can be achieved without a typical inflation cuff. Thus, we were able to demonstrate the PYNQ-Z2 can be used for advanced signal processing as well as feature extraction and continuous health monitoring.[12].

B. S. Naraayanan et al. (2024) The information presented in this study will show how to apply privacy enhancement technologies within the context of the Internet of Things Edge (IoT Edge) using Number Theoretic Transforms (NTT) on PYNQ-Z2 boards. Since NTTs form the basis for many of the Post-Quantum Cryptographic Systems (PQC) being developed and used today (particularly those using lattice-based cryptography), determining the performance capability for using NTT on Zynq SoC was an important aspect to investigate using this research. This study also showed the use of hardware acceleration for NTTs improves performance, which means higher throughput of cryptographic primitives will be possible when hardware acceleration is utilized. The results of this study support the development of Privacy-Preserving and Future-Proofed infrastructures for IoT. [13].

V. C. Johnson et al. (2023) The PYNQ-Z2 evaluation document reviews its hardware acceleration features and possibilities, as well as providing detailed instructions on programming and using the PYNQ-Z2 with Python. It explains how this device operates with its programmable logic device (PLD) in terms of using the Python language. Based on this comprehensive evaluation, the authors feel confident that the PYNQ ecosystem offers many effective options for computer engineers when creating different kinds of high-performance hardware accelerators. Furthermore, the PYNQ-Z2 Evaluation Document provides sufficient information for engineers to base their decision-making process on when using the Python-on-Chip frameworks to create quick prototypes or to utilize other flexible programming options in developing those prototypes [14].

P. Guddati et al. (2024) In Prasad et al. (2021)'s work, researchers illustrated how a proposed deep convolutional neural network (DCNN) model is implemented and evaluated for the fully automated computerised analysis of Chest X-Ray (CXR) images to detect tuberculous infections and pneumonia. Details are revealed regarding how to effectively optimise the DCNN architecture for implementation onto an FPGA while maintaining high accuracy in the diagnosis of both conditions. Ultimately, many portable AI-based diagnostic applications exist that will allow for the delivery of healthcare services to patients who reside in remote locations. The research that Prasad et al. (2021) have presented here will serve as a major step towards accomplishing the continued improvement of medical imaging solutions by enabling the utilisation of embedded hardware with large amounts of imaging data in their development [15].

Summary of Literature Survey

- When conducting an examination of the literature for this report two technologies were identified that are developing at the same time and speed. These two technologies are intended to advance the capacity for complex computations on Embedded Hardware and to advance the development of secure Protocols for protecting digital content at a sufficient speed and sophistication level. Through the integration of high-level Programming (HL/P) with custom Hardware Logic (HL), the PYNQ-Z2 FPGA has become an important resource for researchers involved in developing Prototypes of Energy-Efficient and High-Performance Solutions. In this section of Report, separate Research Studies that focused on either increasing the Performance or Time/Speed at which both Signal Processing and Deep Learning Processes can be performed are discussed. Additionally, we find that in addition to increasing processing times/speeds, new and innovative methods of creating Strong Algorithms to encrypt and protect data are now becoming available.
- The PYNQ-Z2 has been established as one of many platforms that offer a new and unique development environment for the purpose of accelerating Deep Learning through the use of Signal Processing Technology.
- CNN and Vision Implementation: Building on work that showed such complex CNNs, such as ResNet20, could run on FPGAs for image classification tasks, this line of research provides strong evidence that deep learning technology can be efficiently ported to embedded FPGA platforms, reaching remarkable speedups and efficiency [1][15]. FPGA technology is also making its mark in health, particularly in cardiac signal analysis.
- Many works have demonstrated real-time ECG classification for patient condition

monitoring and predictive analytics about strokes [7,11]. Researchers highlight that FPGAs are capable of providing a time-sensitive processing platform for biosignals emanating from various industries [12].

Advanced Digital Media Security and Cryptography:

- Due to the increasing demand for timely security with the development of IoT technologies and their respective processing limitations, current research focuses on fast yet secure encryption algorithms. Co-simulation of existing industry encryption algorithms, such as AES-128, with the associated hardware needed to support these algorithms ensures ciphers are fast and secure [2].
- Chaotic cryptography represents another big part of current research. It includes the ability to take advantage of the chaotic nature of these systems for creating an extremely secure, lightweight encryption key and an encrypted key stream for the purpose of scrambling images. The method relies on the principles of chaos theory to develop a secure, lightweight encryption algorithm well-suited to devices at the network "edge" [3][5].
- In addition to the above methods of encryption, researchers explore innovative methods inspired by molecular biology, such as the use of DNA subsequence operations for devising algorithms that enable image encryption [8]. The encryption algorithms derived from such methods are very complex, yielding a very large number of encryption keys and thus being highly resistant to advanced cryptographic attacks.
- Further, additional comprehensive techniques are being developed that provide various layers of protection by integrating cryptography and steganography into sensitive text and image data for authentication and security purposes [6].

CHAPTER 3

DESIGN AND IMPLEMENTATION

The configuration is meant to encrypt and decrypt images based on AES encryption on the Pynq Z2 board. More interesting, though, is the way it divides the work between the software code that runs on the ARM processor and custom hardware implemented in the FPGA fabric. This results in images being processed much quicker than if all of this had been done in software, really sort of the entire reason for using an FPGA to begin with.

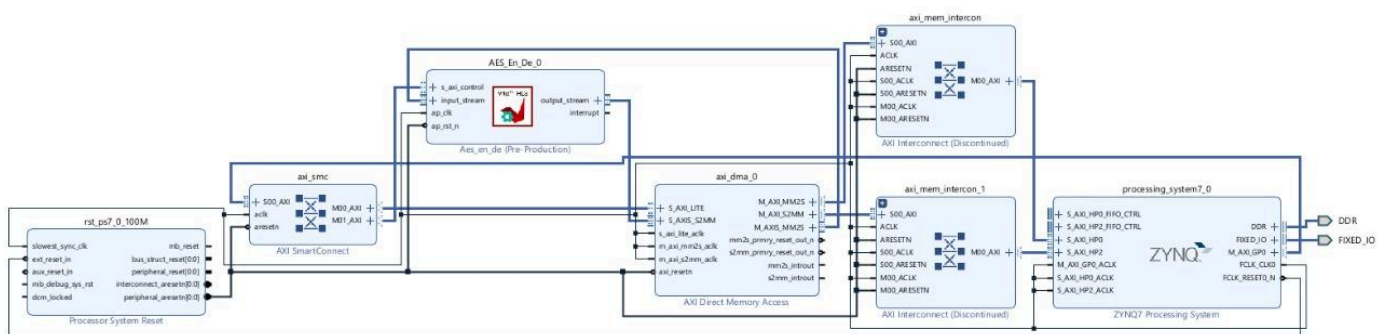


Fig 3.1 - Block diagram of Proposed Methodology

3.1 Block Diagram Components

3.1.1 Processor System Reset (rst_ps7_0_100M)

- This module resides on the left-hand side farthest from the rest and manages all the reset signals for the system. This module lays out the order in which everything should be initiated, when the board is booted or needs to be restarted. Coming hotfooting in with commands to the processor, it sends syncs reset times, external resets, auxiliary resets, debug resets and checks for a stable clock via dcm_locked. If it doesn't work, the system may load up in a weird state or even just crash out.

3.1.2 Processing System 7 (processing_system7_0)

- Looking at the Zynq chip, you'll notice it contains an ARM Cortex-A9 processor in the PS7 block, that's basically where our Python script is running. The High Level oversees the overall activity of the FPGA and provides an interface for the User Interface (UI) designed to

load images and perform all in between operations. The processor uses multiple AXI channels to communicate with the FPGA fabric. The AXI channels are all standard buses, allowing for the transfer of data between different components. Essentially when the processor wants to communicate with the FPGA, it will send instructions via the M_AXI channel, while S_AXI allows for data coming from the FPGA back to the processor. An additional signal used between the processor and FPGA is the IRQ signal, which provides notification to the processor of events such as the completion of encryption tasks.

- The PYNQ-Z2 board uses a Xilinx Zynq-7000 SoC (System-on-Chip) called XC7Z020-1CLG400C. It has two (dual) Core ARM Cortex A9 CPUs with a clock speed of 650 MHz and a very large amount of programmable logic (PL) built into the SoC. Therefore, its architecture makes it particularly suitable for applications that require very fast throughput such as cryptographic image processing since it has 13,300 logic slices, 630 Kilobytes_(KB) of Block_RAM, and 220 Digital Signal Processing (DSP) slices.
- The PS contains the software portion of the embedded system that includes both the Linux Operating System and Python Application Layer. The PS handles many of the high-level tasks, including loading images from files, controlling the graphical user interface, and setting up the overall system. The processor interacts with the FPGA via the AXI interface which defines the standard bus architecture used to transfer information between components in the embedded platform.

3.1.3 AXI SmartConnect (axi_smc)

- Looking at the SmartConnect, think of it as a traffic controller for data. It takes AXI signals from the processor, and sends them to the right place in the FPGA. It's fed by the S00_AXI from the PS7 block, and sends its output through M00_AXI. Coming hotfooting into a system with lots of components all wanting to talk at the same time, the SmartConnect sorts out the priority battle so that data doesn't get stuck in the pipes. There's also the occasional need for clock domain crossing, but in our design, all the components run off the same 100 MHz clock.
- A communication system using AXI4-based Communication and the AXI SmartConnect module, SmartConnect modules manage the traffic on your chip, directing all of the chip's signals either from your CPU or directly to the FPGA. SmartConnect also allows you to prioritize signals to prevent bottlenecked data traffic or overload.
- AXI (Advanced eXtensible Interface) Direct Memory Access (DMA) is a method that

eliminates the delays associated with moving data in a serial fashion; instead, AXI allows you to efficiently transfer data. AXI (Advanced eXtensible Interface) DMA allows the transfer of large quantities of data from the DDR memory to the FPGA using the AXI interface(M_AXI_S2MM) and then back (to the M_AXI_MM2S) to the Memory using a separate AXI interface.

- Performance-Oriented Ports: The design of the AXI system includes S_AXI_HP0 and S_AXI_HP1 ports, which provide separate low-latency paths to the various modules used in the system to transfer the processed data to and from the FPGA without creating a bottleneck on the main memory controller.

3.1.4 AES Encryption/Decryption Core (AES_En_De_0)

- The 128-bit block of data and the 128-bit key go through the input_stream, when the encryption happens. The encrypted or decrypted data is sent out via the output_stream. Coming from the outside, you control what's going on with the s_axi_control, and need to sync with the internal clock ap_clk and reset ap_rst_n.
- The key is dynamically scaled and passed through the well-known SubBytes, ShiftRows and MixColumns algorithm, when the encryption process hits the S-box stage. Coming into the HLS code, it's checking for a change in the key. Well-known for its pipelining, this block enables a new piece of data to be fed in while the previous one is still being processed, it's essentially what gives AES-256 its speed.

3.1.5 AXI DMA (axi_dma_0)

- Copying byte by byte is essentially a nightmare, that's what the DMA is for, when transferring large amounts of image data back and forth between the processor's memory and the FPGA.
- Well-known as the Digital Movement Accelerator, the DMA on our board now has its own set of channels for memory-to-memory transfers. The M_AXI_MM2S channel takes in data from DDR memory and sends it to the FPGA, and the M_AXI_S2MM does the opposite, sending data from the FPGA back to memory. The S_AXI_LITE control interface is where our Python code tells the DMA what to read, write and the quantity of bytes to transfer. The mm2s_introut and s2mm_introut signals send us a message when the transfers are complete.

3.1.6 AXI Interconnects (axi_mem_intercon and axi_mem_intercon_1)

- Speaking of interconnects in AXI4, axi_mem_intercon and axi_mem_intercon_1 are

basically advanced SmartConnects that can handle multiple masters and slaves. The `axi_mem_intercon` is used to hook up the read side of the DMA to the DDR memory controller, and `axi_mem_intercon_1` looks after the write side. They're equipped to do address decoding, and also can widen the data bus if necessary, stopping collisions between transactions. Coming from the DMA, these interconnects have `S00_AXI` inputs and send `M00_AXI` signals to the rest of the system.

3.1.7 Processing System 7 (Second Instance - `processing_system7_0`)

- When you first glance at the design diagram, you might do a double-take. There isn't a real second processor here-just two views of the same Zynq-7000 inside the Vivado project. I mistook it for a duplicate at first, until it clicked: the tool isn't doubling up processors, it's presenting the chip from two angles. Think of a house blueprint shown twice-once to lay out doors and windows, and again to map the plumbing and wiring. The second block, `processing_system7_0`, is that second perspective. It highlights the chip's key data paths-the High-Performance AXI ports, namely `S_AXI_HP0` and `S_AXI_HP1`.
- For our project, these ports aren't just technical details on a diagram; they're the reason our whole system can keep up. FPGAs are extremely fast, but to use that incredible speed, they must be supplied with data at the same rate. The HP ports provide what are essentially dedicated freight lanes for transporting the image data between the custom encryption logic and the main memory controller. Without these ports, the hardware accelerator would be like a Formula One race car stuck in bumper-to-bumper traffic - it has incredible speed capabilities but nowhere to go. When we encrypt a medical image, the enormous amount of data that comprises this file can flow rapidly via the high-performance ports to the FPGA, where it is encrypted in real-time (in the blink of an eye) and returned via the same path back to the host system. This architectural design is critical to turning theoretical performance enhancements into working systems.

3.1.8 How Data Actually Flows Through the System

- Next, we need to describe how an image would be affected once we enable encryption on the image. In this section of the application, the Python script takes the image file and will pad it to the next closest multiple of 16 bytes, after which the image chunks are generated as 2MB chunk sizes by slicing the original image file. This triggers the initialization of the Direct Memory Access buffers for the chunks. The encryption key and all control parameters will now be passed through the interface with the AES core using the `s_axi_control` interface. The

Direct Memory Access controller will also be configured to pull data from the input buffer stored in the DDR memory during the process of creating the image chunks. Once the MM2S Direct Memory Access channel pulls the data from RAM, it transfers the data through `axi_mem_intercon` and sends it to the `input_stream` of the AES Core.

- The AES hardware begins its rounds of heavyweight substitution and permutation operations, as we discussed above, working through the data. The encrypted blocks are spit out of the `output_stream` and into the S2MM channel of the DMA.
- The S2MM side of the DMA writes the results back into the DDR memory via `axi_mem_intercon_1`. Once completed, the processor receives an interrupt, which makes Python go back and retrieve the encrypted data from the output buffer, move to the next chunk if available, or put everything back together into the final encrypted image. The same idea applies to decryption, except the mode bit would be reversed, so that the AES core would perform the operations in the opposite order.

3.1.9 Why This Architecture Makes Sense

- Splitting it in this manner exploits the strengths of each part: the ARM core does a really good job at the high level logic, file I/O, and Python (language) interpretation, but it is not the best with the repetitive math required with respect to encryption. The FPGA can implement custom hardware to do AES transforms much, much faster than any general-purpose processor.
- The DMA is significant because it relieves the processor of having to babysit each data transfer. Once the DMA transfer is initiated, it simply transfers data on its own while the processor meanwhile does something else or waits for it to complete (and the same for the next transfer). The multiple AXI interconnects eliminate procedure bottlenecks- reads and writes can happen at the same time along different routes.
- The entire system runs at 100 MHz, which doesn't sound like it would be crazy fast, but keep in mind that the AES core will handle 128 bits per cycle as it is pipelined. That will be faster than what you would get from a two-processor implementation, especially for the big images this project deals with.

3.2 Hardware & Software Details

In this project, we implemented a flexible FPGA-based approach to image cryptography using the PYNQ-Z2 board, leveraging the parallel processing power of its Zynq-7000 SoC. We designed the cryptographic algorithm in C++, synthesized it using Vivado HLS, and deployed the

resulting IP as a hardware overlay in Vivado. The final bitstream was loaded onto the board and accessed via Jupyter Notebook using the PYNQ Linux environment, enabling seamless interaction with the FPGA through Python. This setup offered high performance, reconfigurability, and user-friendly integration for real-time image encryption and decryption.

The Hardware components used in this project are:

3.2.1 PYNQ-Z2 FPGA Development Board

The core hardware used in this project is the PYNQ-Z2, a development board built around the Xilinx Zynq-7000 SoC, specifically the XC7Z020-1CLG400C. This SoC combines a dual-core ARM Cortex-A9 processor (operating at 650 MHz) with a sizable programmable logic (PL) section. The PL includes 13,300 logic slices, 630 KB of Block RAM, and 220 DSP slices, making it highly suitable for implementing parallelizable tasks like image encryption.

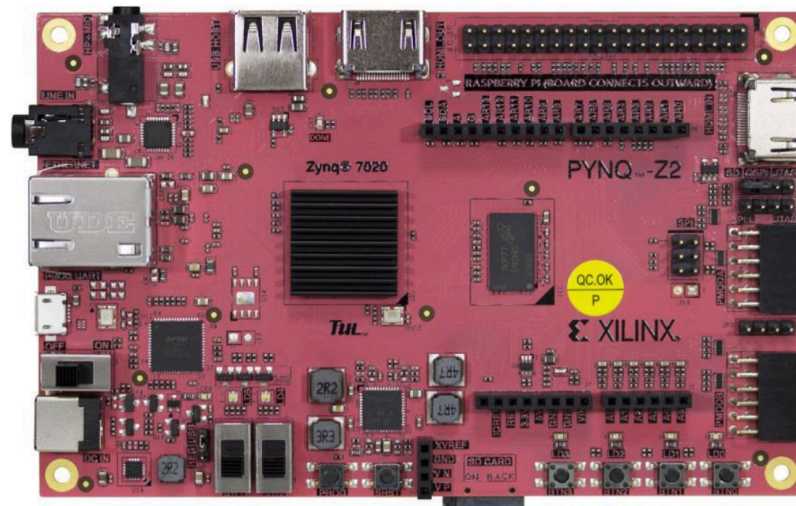


FIG 3.2 - PYNQ Z2 FPGA DEVELOPMENT BOARD

The board is equipped with 512 MB of DDR3 RAM, connected via a 16-bit bus running at Up to 1050 Mbps, and also includes 16 MB of Quad-SPI Flash memory and a MicroSD slot. It supports multiple power options: Micro-USB, 7V–15V external power supply, or a battery pack, configured via jumper J9. For connectivity, the board features Gigabit Ethernet, USB 2.0 OTG, and Micro-USB for UART and JTAG programming.

Crucially for image cryptography, the PYNQ-Z2 offers two HDMI ports (input and output) connected directly to the FPGA fabric, enabling real-time image data transfer. Other features include an ADAU1761 Audio Codec, push buttons, slide switches, LEDs (including RGB LEDs),

and expandable headers compatible with Pmod, Arduino shields, and Raspberry Pi GPIO.

This hardware versatility, combined with native support for the open-source PYNQ framework, makes the PYNQ-Z2 ideal for embedded cryptographic applications, real-time signal processing, and reconfigurable computing.

3.2.2 Power

The PYNQ-Z2 can be powered from the Micro-USB port (J8), an external power supply, or a battery. The power source is selected by setting jumper J9 (near SW1) to USB or REG (External power REGulator/Battery). The MicroUSB port connects to a standard USB port and should provide enough power for most designs. More demanding applications may require more power than the USB port can provide. When more power is required, an external power regulator (coax, center-positive 2.1mm internal-diameter plug) can be connected to the power jack (DC1). The board supports 7 VDC to 15 VDC (12V recommended). A battery can also be used to power the PYNQ-Z2 by attaching the positive terminal to the “VIN” pin on the Arduino J7 connector (with jumper J9 set to REG). The negative terminal can be connected to one of the pins labeled GND on J7.

3.2.3 Boot Selection Mode

The PYNQ-Z2 supports MicroSD, Quad SPI Flash, and JTAG boot modes. The boot mode is selected using the Mode jumper (JP1). TO select the boot mode, move the jumper to the appropriate position as indicated by the label on the board.

LED (Light Emitting Diode): An LED is a semiconductor device that emits light when an electric current passes through it. It is used for indication purposes in circuits, such as showing power status or signaling output. LEDs come in various colors and are energy-efficient, long-lasting, and fast- switching. Push Button: A push button is a simple mechanical switch that temporarily completes or breaks an electrical circuit when pressed. It is commonly used as a user input device in electronic projects to trigger events like turning a device on/off or sending commands to a microcontroller.

3.2.4 Clock Sources

The PYNQ-Z2 provides a 50 MHz clock to the Zynq PS_CLK input, which is used to generate the clocks for each of the PS subsystems. The 50 MHz input allows the processor to operate at a maximum frequency of 650 MHz. The PYNQ-Z2 also has an external 125 MHz reference clock connected to pin H16 of the PL. The external reference clock allows the PL to be used independently of the PS. Outlines the clocking scheme used on the PYNQ-Z2. Note that the

reference clock output from the Ethernet PHY is used as the 125 MHz reference clock to the PL. When the PHYRSTB signal is driven low, the Ethernet PHY (U5) is held in reset disabling the CLK125.

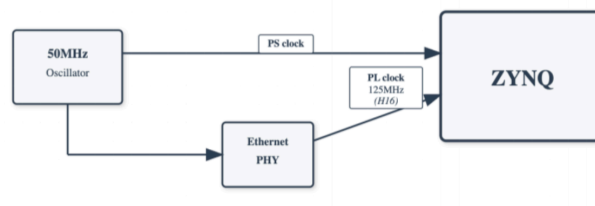


FIG 3.3 - ZYNQ Clock Distribution Architecture

3.2.5 Dram

The PYNQ-Z2 includes a Micron 256Mx16 DDR3 memory (MT41K512M16HA-125:A) creating a single rank, 16-bit wide interface with a total capacity of 512MB. The DDR3 is connected to the hard memory controller in the Processor Subsystem (PS). The PS incorporates an AXI memory port interface, a DDR controller, the associated PHY, and a dedicated I/O bank. DDR3 memory interface supports speeds of up to 525 MHz/1050 Mbps on the PYNQ-Z2 board. For best DDR3 performance, DRAM training is enabled for write leveling, read gate, and read data eye options in the PS Configuration Tool in the Xilinx tools. Training is done dynamically by the controller to account for board delays, process variations and thermal drift. The PYNQ-Z2 board files (see section 2) contain the configuration for the DRAM controller which includes optimum starting values for the training process taking into account PCB and trace delays (propagation delays) for the memory signals board delays are specified for each of the byte groups. These parameters are specific to the board, and were calculated from the reported PCB trace lengths. The DQS to CLK Delay and Board Delay values are both calculated specific to the PYNQ-Z2 memory interface PCB design.

3.2.6 Quad SPI Flash

The Quart-SPI serial NOR flash memory manufactured by Spansion known as S25FL128S located on the PYNQ-Z2 has 16MB of memory. The flash memory is rated 104MHz bus speeds and operates in x1, x2, and x4 mode. The Zynq is capable of configuring the flashed memory in Quad-SPI mode with speeds near 100 MHz to achieve data rates up to 400Mbps. The flash memory operates at 3.3V, and operates as a means of initializing and booting the Processing System (PS) Subsystem and configuring the Programmable Logic (PL) Subsystem. It can store

code and/or data in a non-volatile memory. The SPI flash connects directly to the Zynq-7000 SoC via the Quad SPI interface, requiring connections to MIO pins [1:6,8], as specified in the Zynq datasheet. Quad-SPI feedback mode is used, thus `qspi_sclk_fb_out/MIO[8]` is left to freely toggle and is connected only to a 20K pull-up resistor to 3.3V. This allows a Quad SPI clock frequency greater than `FQSPICLK2`

3.2.7 USB Host

The PYNQ-Z2 includes a TI TUSB1210 USB 2.0 PHY with an 8-bit ULPI interface connected to the Zynq PS USB 0 controller (MIO[28-39]). The PHY features a HS-USB Physical Front-End supporting speeds of up to 480 Mbs. The USB interface is configured to act as an embedded host. USB OTG and USB device modes are not supported. One of the Zynq PS USB controllers can be connected to the appropriate MIO pins to control the USB port.

3.2.8 Micro SD

The PYNQ-Z2 has a MicroSD slot (SD1). An SD card can be used to boot the board, or for applications that require non-volatile external memory storage. The PS IOP controller SDIO 0 is wired to this port via MIO[40-47]. The pinout can be seen in Table 7.1. The peripheral controller supports SDIO host mode with 1-bit and 4-bit SD transfer modes. SPI mode is not supported. The Zynq PS UART control can be connected to the appropriate MIO pins to control the MicroSD port. The maximum clock frequency is 50 MHz which supports both low-speed and high-speed cards. A Class 4 MicroSD card or better is recommended.

3.2.9 Ethernet PHY

The PYNQ-Z2 has a Realtek RTL8211E-VL PHY supporting 10/100/1000 Ethernet. The PHY is connected to the Zynq RGMII controller. The auxiliary interrupt (INTB) and reset (PHYRST) signals connect to MIO pins MIO10 and MIO9, respectively. One of the Zynq PS Ethernet controllers can be connected to the appropriate MIO pins to control the Ethernet port. The Zynq does not need to be configured for the PHY to establish a connection. After power-up the PHY starts with Auto Negotiation enabled, advertising 10/100/1000 link speeds and full duplex. The PHY will automatically establish a link if there is an Ethernet-capable partner connected. There are two status LEDs on the RJ-45 connector that indicate traffic activity and link Status.

The Link LED, located on the right and colored green, blinks when the device is transmitting or receiving data. The Act LED, positioned on the left and colored yellow, blinks to indicate activity on its port and remains off when no link is established.

The Software components used in this project are:

The image encryption and decryption system on the PYNQ-Z2 leverages a combination of hardware acceleration and high-level software control. The Vivado Design Suite allows to design, synthesize, and implement cryptographic logic on an FPGA by using Vivado HLS (High-Level Synthesis). The tools employed in conjunction with VIVADO and various versions of Python, as they relate to Jupyter Notebook integration, will provide the user with everything they require for designing and building photo-encryption devices. The toolchain options available within Python will enable users to manipulate images and measure performance while at the same time allowing users to interact with their applications.

Ultimately, the goal of utilizing multiple toolchain options within Python will be to provide a single platform for developing and implementing a photo encryption device that provides a simple method for end users to design or optimise their own encryption algorithms by way of a graphical user interface (GUI) with a corresponding software program, allowing users to formulate accurate compiling commands for their devices, control the operation of their hardware during execution, and communicate between Jupyter and PYNQ-based processors in an effective manner.

Most of the hard work related to such as image operation performance monitoring and ease of use will be performed through the use of various Python Libraries (as they take a more software-based view to their solutions).

3.2.10 Vivado Design Suite

The Vivado Design Suite is a software that is used to design and manage the hardware design for Zynq-7000 SoC on the PYNQ-Z2. The main use of Vivado is in the synthesis of RTL designs, the creation of the IP integrator block diagrams to be used by the processors, and the generation of the bitstreams that configure the FPGAs PL. In this project, Vivado is used to include custom blocks that perform encryption and decryption (designed in HLS) into the block design. Vivado also creates the necessary AXI interconnects that allow the PS portion to communicate with the PL, enabling communication between the ARM cores. After the implementation is complete, and the bitstream has been generated, the design is exported to the PYNQ requirement (to be used within PYNQ).

System Requirements:

Operating System: Windows 10 (64-bit) or Linux (RHEL/CentOS 7.6+/Ubuntu 18.04/20.04LTS)

RAM: Minimum 8 GB (16 GB or more recommended)

Disk Space: At least 50 GB free

Processor: Multi-core CPU (Intel i5/i7 or AMD Ryzen equivalent recommended)

Additional: Java runtime, hardware USB drivers for JTAG/debug tools

3.2.11 Vivado High Level Synthesis(HLS)

Vivado HLS enables the development of custom hardware blocks using high-level C/C++ or OpenCL descriptions, which are synthesized into RTL. In this project, image cryptography algorithms such as AES or custom ciphers are implemented as C/C++ functions and then synthesized using HLS into hardware IP cores. These cores are optimized for latency and resource utilization, and then packaged into IPs to be used in the Vivado block design. This HLS-driven design approach significantly shortens development time and allows for iterative design and testing of the encryption algorithm.

System Requirements: Same as Vivado Design Suite (integrated within the same installation package)

3.2.12 PYNQ Framework

The PYNQ (Python Productivity for Zynq) framework enables the deployment of hardware-accelerated applications using high-level Python interfaces. In this project, once the encryption and decryption IPs are integrated and deployed onto the FPGA, PYNQ is used to interact with them via memory-mapped registers. The overlays created from Vivado bitstreams are loaded using Python, and data (image buffers) are sent from the ARM core to the PL via AXI DMA or AXI-Stream interfaces. The PYNQ framework handles this data movement seamlessly, enabling real-time encryption/decryption with minimal software overhead.

System Requirements: Pre-installed on the PYNQ-Z2 board's SD card image (Ubuntu-based Linux OS)

Host PC: Any modern computer with a web browser for accessing the PYNQ board over the network

Recommended Browser: Google Chrome, Firefox, or Microsoft Edge

3.2.13 Jupyter Notebook

Jupyter Notebook provides the user interface for software control and visualization of the image encryption pipeline. It acts as the main development and testing environment for this project. Python scripts are written to load the bitstream overlay, configure the hardware IPs, transfer image data to the FPGA, and retrieve the encrypted or decrypted results. The notebook is also used to validate the functionality, perform testing with various image formats, and visualize before-and-after images using plotting tools. Additionally, the notebook supports easy debugging and documentation through markdown and interactive cell execution.

System Requirements: Hosted on the PYNQ-Z2 board or run locally on a PC with Python and Jupyter installed.

Compatible browser: Chrome, Firefox, or Edge

3.2.14 Python & Support Libraries

The Python ecosystem plays a critical role in managing software-side operations, including file I/O, image manipulation, and performance measurement.

Key libraries include:

NumPy: Array operations, and buffer management.

OpenCV or Pillow: Image processing and format conversions.

Matplotlib: Visualization and debugging.

PYNQ Library: This provides a Python-based interface to the hardware IPs and PL memory. The flow using these libraries is executed within the ARM-based Linux OS running on the PS; fast, flexible development workflows without re-synthesis of the hardware are thus supported. With these two libraries, the complete encrypt/decrypt software-hardware loop can be executed interactively, with great flexibility in trying out multiple image files and algorithms.

System Requirements:

Python Version: 3.6 or higher (Python 3.8 recommended)

Libraries: NumPy, Pillow or OpenCV, Matplotlib, PYNQ Python package

Installation: Use pip or conda (preferably within a virtual environment)

CHAPTER 4

FLOWCHART

Introduction

We started in Vitis HLS, which feels more like traditional programming. We wrote our LED cipher in C++—something we knew well—but then we added special hints to the compiler called "pragmas." These aren't commands; they're more like suggestions that tell the tool how we want the hardware to behave: "This port should connect here," or "These operations can run in parallel." When we hit "Run C Synthesis," it felt a bit like watching a translation happen—our familiar C++ logic was being transformed into the actual logic gates and flip-flops that would make up our custom encryption engine. An output IP core is essentially a reusable hardware module packaged with a set of clean interfaces so that it can be rapidly integrated into a larger system.

The vehicle for actually integrating the various systems was Vivado. With the block diagram editor in Vivado, we were able to drag-and-drop our encryption IP core, the Zynq processing system (the brain of the board), a DMA controller (our designated data mover), plus all of the necessary interconnects. The AXI interfaces we were working with have control, data, and high performance stream master/slave ports; taking time to make sure every interface is properly wired is critical and missing one wire could thwart the entire system validation process. Once all of the AXI interfaces have passed all of their checks and are confirmed validated, the last step of building the system is to create the Bitstream file. The Bitstream file uses a protocol that represents the internal configuration of the FPGA; it allows you to build your target system by providing exact software implementations of the intended design.

Upon finishing this final phase of development, the hardware is activated; specifically, the Bitstream is "burned" onto the PYNQ board, so now what was once just an empty chip can now function as an Encryption Accelerator FPGA. This allows you to run a Jupyter Notebook with your own Python code and communicate directly with the hardware as though it were any other part of a computer, sending your image buffer to the encryption core using AXI Control Registers and moving your image data from system memory to the Hardware Module using DMA as part of the same Python program. We can see the encrypted image assemble and then fetch the exact same picture in decrypted form—all done with just a few lines of Python.

This entire flow—from high-level code, through system design, to interactive hardware—shows how modern FPGA development has become accessible. It still requires careful thinking and debugging, but the tools let you focus on the architecture and the problem you're solving, rather than getting lost in the lowest-level details.

- The Xilinx Vivado High-Level Synthesis (HLS) toolchain was used as part of the development process to allow the encryption algorithm to be developed in C++ and then converted into an optimized hardware circuit. This approach allows the developer to eliminate the complexity of writing RTL code manually, therefore reducing development time.
- When using Vitis HLS, the C++ code also has pragmas (compiler directives) added to help with mapping the software logic to the physical hardware (gate) implementation.
- Interface Definition: The Interface Definitions for both the AXI Stream and AXI Lite array are designated through pragmas.
- Optimization: Optimizing the circuit using specific options in Vitis HLS will lead to a smaller and faster circuit; for instance, implementing Pipelining will allow the next data to enter the core before the previous message has finished processing.
- After writing the C++ code, the final step is to run C Synthesis in order to convert the high-level C++ language to a Register Transfer Level (RTL) hardware description. When the synthesis is complete, an IP package will be generated for use in designing your block designs using Vivado.

Methodology and Interface Definition

- The cryptographic core implementation was completed via Xilinx Vitis High-Level Synthesis (HLS). Xilinx Vitis HLS was selected for its ability to abstract the hardware design process. By using HLS, AES-128 can be modeled with C++ and synthesized into a Register Transfer Level (RTL) netlist prior to developing the physical hardware design. Unlike previous designs, we completed the architectural constraints for integration into the Zynq-7000 System on Chip (SoC) using HLS pragmas—an extremely valuable step in successfully integrating the core into the Zynq-7000 SoC.
- Modeling the algorithmic basis for the encryption logic as a bit-accurate C++ modeling code first created a Golden Reference model to verify functional correctness for the substitution and permutation layers prior to committing to any hardware solutions.
- A key element of the design was determining how to interface the C++ Function Arguments with the AXI4 Hardware Protocol to enable interaction with the ARM Processor and DDR

memory.

- Control Plane (s_axilite): As part of the scalar control protocol, the Start, Stop and ap_done interrupt controls were mapped to an AXI4-Lite Interface and placed into a memory mapped register block, allowing a Python driver within the Processing System (PS) to configure the core and monitor its status.
- Data Plane (axis): Inputs and outputs for the image arrays were mapped to AXI4-Stream Interfaces. Unlike memory mapped access, the AXI4-Stream Protocol provides for Burst-Oriented data transfers which is instrumental in reducing address overhead and maximizing performance.

Optimization Directives and RTL Generation

- Certain optimization directives were used, after which the default sequential execution of the code was translated into its parallel hardware architecture realization for real-time performance. During synthesis, these went through several iterative refinements as a balance of resource usage and timing constraints was sought.
- Pipeline Optimization: The inherent iterative structure of AES was optimized by applying the `#pragma HLS PIPELINE` directive to the main processing loop. The scheduler was instructed to lower the Initiation Interval (II) to one; therefore, the core can take a new 128-bit data block on each clock cycle since it will be overlapping the execution of the instruction(s).
- Identifying Memory Partitioning: The largest potential bottleneck was the lookup table of the SubBytes transformation. The table had been stored in BRAM and thus had limited read accesses. Applying the `#pragma HLS ARRAY_PARTITION` directive allowed a separation into individual registers in the BRAM. This allowed the user to perform 16 reads for every clock cycle and perform parallel processing on the entire state matrix.
- Fully Unrolling the Loops: The inner loops in MixColumns and AddRoundKey were completely unrolled using the `#pragma HLS UNROLL`. This leaves the synthesis tool with no other option but to instantiate parallel hardware units used in the Galois Field multiplication, instead of reusing only one operator to decrease the overall latency of each round of encryption.

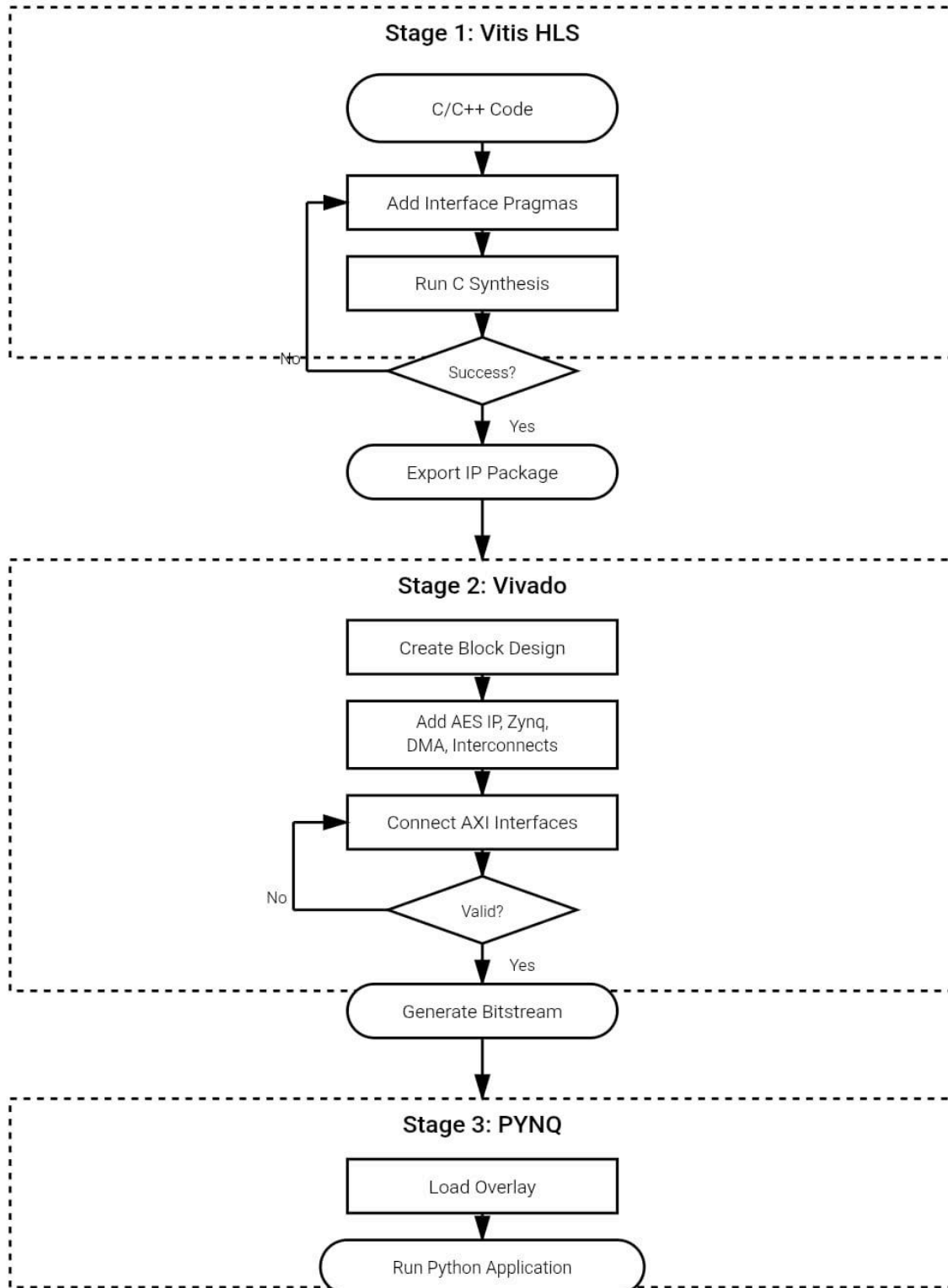


Fig 4.1 - Development Process of Image Cryptography on PYNQ-Z2

4.1 Development Workflow

4.1.1 Vitis HLS

The development begins with writing the AES encryption and decryption algorithms in C/C++ code. Pragmas for the type of interfaces are used to define AXI Stream and AXI Lite interfaces for communicating with hardware. Once the code has been written, it will undergo a process called C Synthesis, during which the compiler converts the high-level language into RTL (Register Transfer Level) hardware. If the synthesis does not succeed, you will have to debug and correct the coding errors before it can proceed again. If the synthesis is successful, you will have an IP_package (or .zip archive) containing your design, which you can download to integrate into your own projects.

4.1.2 Vivado

A block design in the Vivado environment contains the AES IP component created with HLS in the IP repository. The primary components of the design consist of a Zynq Processing System (PS), an ARM processor based on a standard ARM processor architecture, a custom AES IP core, and an AXI DMA Controller for data transfer between the processor and programmable logic (PL) at high speeds. This design will contain AXI Interconnects used to connect the various parts of the design. Each design component will also have an AXI Interface that provides a pathway from the processor to the PL. The design validated within Vivado will verify problems such as connection errors, mismatched widths, and missing ports, as well as correct any errors prior to completing the validation of the design. If a successful validation is achieved, the design's bit-stream, Bit File (.bit), and Handoff File (.hwh) for use with hardware will be generated through synthesis, implementation, and design generation.

4.1.3 PYNQ

We copy the bitstream and hardware description files over to the board when using the PYNQ-Z2 board. Coming from Jupyter Notebook, we then load the overlay that contains a .bit and .hwh file which configures the FPGA fabric with our AES design. Our python application is then executed and uses the PYNQ framework to communicate with the hardware. The framework will handle the DMA transfers, configurable registers, send through the hardware the image data that we want to encrypt/decrypt, and get the results back. This is the complete three-stage workflow methodology starting all the way from algorithmic C code to functional hardware acceleration at high-level programming using Python.

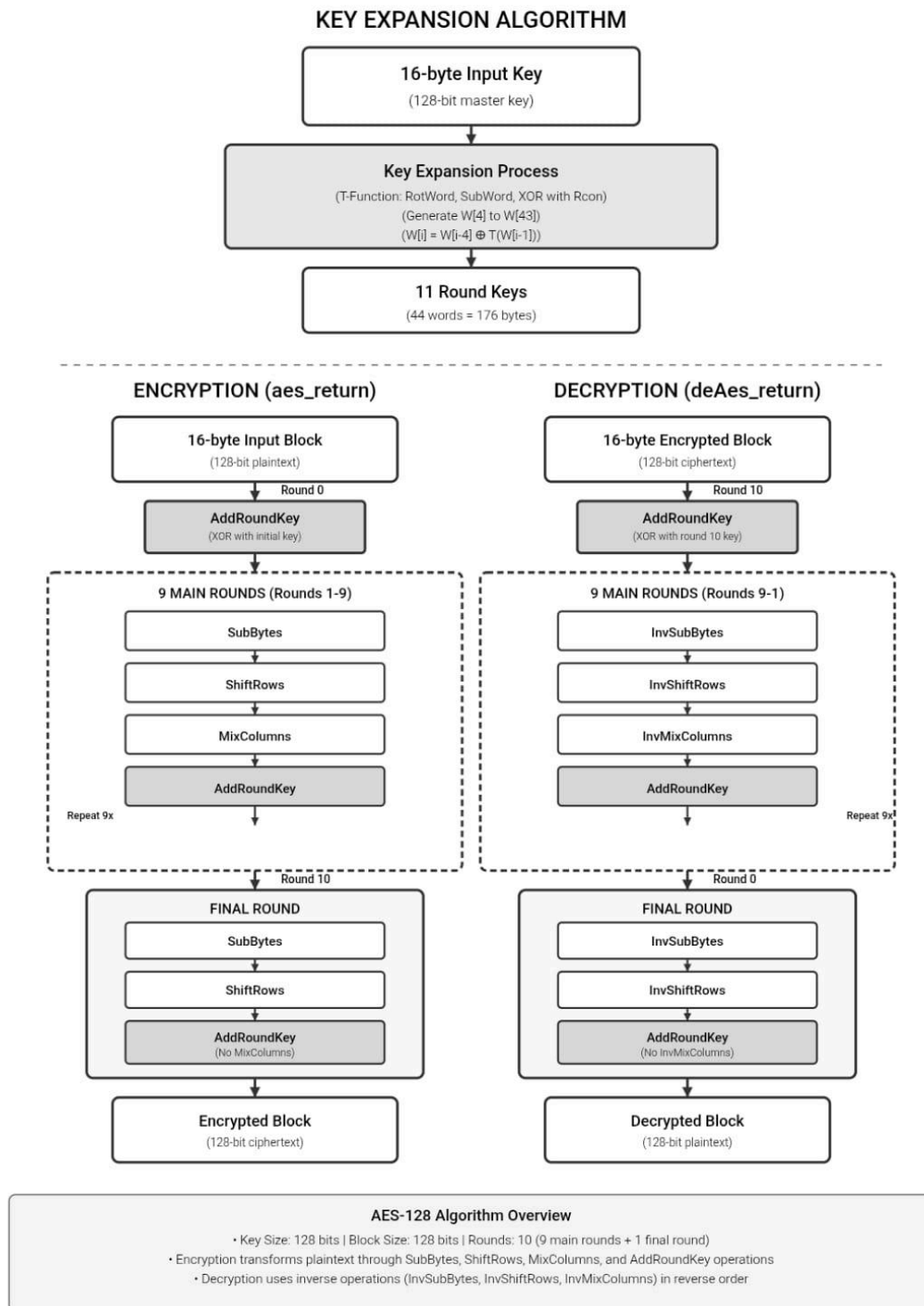


Fig 4.2 - AES-128 Flowchart

4.2 AES-128 Algorithm Workflow

4.2.1 Key Expansion Algorithm

The process begins with a 16-byte input key (128-bit master key). The Key Expansion Process applies transformation functions including RotWord, SubWord, and XOR with Rcon (round constant) to generate extended key schedule. This produces 11 Round Keys totaling 44 words or 176 bytes, where each subsequent word is computed as $W[i] = W[i-4] \oplus T(W[i-1])$.

4.2.2 Encryption Process (aes_return)

AES encrypts a 16-byte (128-bit) block of plaintext using 10 rounds. The first step (Round 0) is to use the original plaintext and combine it with the secret key (AddRoundKey) using the XOR operation. The transformation of the data happens during rounds 1 through 9. Each of these rounds consists of four operations - SubBytes, ShiftRows, MixColumns, and AddRoundKey. The 10th round completes the transformation by only using SubBytes, ShiftRows, and AddRoundKey, to produce a final encrypted 16-byte block.

4.2.3 Decryption Process (deAes_return)

The process of restoring encrypted text is a reversal of how the initial encryption was completed. Reverse actions are sequentially taken to move back to the raw information starting with each equivalent key. To start the reverse process of AES decryption the encrypted text will need to be decoded first using AddRoundKey using the final round key for round 10. The following steps can now begin in reverse from round nine through round one, completing each of the reverse actions for each of the eight rounds. The first decrypting process in round one is InvSubBytes followed by InvShiftRows followed by InvMixColumns. The final round uses InvSubBytes and InvShiftRows to decrypt the final round. If everything is completed accurately, then the 16-byte block will contain the original functions of information.

4.2.4 Algorithm Overview

Think of AES-128 like a carefully-designed 10 act play where everything is planned out ahead of time for the sole purpose of converting your readable data into something that looks like complete gibberish or random noise. AES-128 uses a specific size of a block of 128 bits of data to be encrypted with a 128-bit secret key, not just as a password you type in, but a key that will dictate every single movement of the paper throughout its entire journey. Encryption unfolds through a sequence of four distinct, mathematically elegant operations—SubBytes (a non-linear

byte-by-byte substitution using a predefined S-box), ShiftRows (a transposition that cyclically shifts rows of the state array), MixColumns (a linear mixing that diffuses data across columns via matrix multiplication in the Galois field $GF(2^8)$), and AddRoundKey (a simple but crucial XOR with a round-specific key derived from the original)—executed over 9 full rounds followed by a final, slightly simplified 10th round. Decryption isn't merely reversing the sequence; it's executing the exact mathematical inverses—InvSubBytes, InvShiftRows, and InvMixColumns—in reverse order, while AddRoundKey (being its own inverse) elegantly closes each step. This reversible yet computationally diffuse structure ensures that even the smallest change in the input propagates dramatically, providing robust security through a harmonious balance of confusion, diffusion, and algorithmic symmetry.

- **Architectural Specifications:** The system covers the implementation of the AES-128 standard, whose algorithm works with a block size of 128 bits (16 bytes) and a cipher key size of 128 bits. An iterative encryption process is used, composed of 10 rounds of processing. Full sets of transforms are executed in the first nine rounds, while mixing operations are not performed in the last round for efficient decryption.
- **Transformation Layers:** Every round transforms the 128-bit state matrix using four different operations:
- **SubBytes:** a non-linear substitution step whereby each byte is replaced by its value in a pre-calculated S-Box table in order to introduce confusion.
- **ShiftRows:** a transposition step that cyclically shifts each row of the state matrix for avoiding independent column encryption.
- **MixColumns:** A linear diffusion operation in which the columns are mixed together using matrix multiplication in the Galois Field $GF(2^8)$
- **AddRoundKey:** A bitwise XOR of the state with a distinctive Round Key, which is derived from the main cipher key. **Key Schedule:** The 128-bit master key is expanded into a set of eleven 128-bit Round Keys via the Key Expansion algorithm. This process employs non-linear substitutions and round constants for the purpose of making each round use a key that is distinct from a cryptographic perspective—a measure to prevent pattern recognition attacks.

System Advantages

- **High Throughput from Parallelism** - Where FPGAs Outshine Rather than executing instructions sequentially as a General-Purpose Processor (GPP) does, FPGAs exploit an additional layer of Parallelism in space. (i.e. many functions are processed simultaneously). By using loop unrolling on MixColumns and AddRoundKey, the FPGAs can execute

greatly increased quantities of arithmetic computations in parallel, performing many more operations than a GPP in the same time period. This enables astonishingly fewer clock cycles (or time) to process one round of encryption – vastly increasing throughput for applications where large-scale image streaming is present. If you have tested software implementations with similar use cases you will experience vastly improved throughputs with this solution.

- **Decrease in CPU Utilization** - Programmable Logic can offload the computational burdens of Cryptographic Processing from the ARM Cortex A9 processor and allow it to optimize general-purpose processing tasks (i.e. Operating System, networking etc.). This enables your system to stay responsive, even under a high volume of Cryptographic Processing. The biggest problem with pure software implementations has been the inability to provide an uninterrupted or highly responsive experience, where users experienced freezing or lag.
- **Deterministic Latency** - Operating System software relies on a scheduler to determine when to allocate CPU resources. This creates “scheduler jitter” and can cause unpredictable and/ or inconsistent processing times. On the other hand, hardware Intellectual Property (IP) cores will execute with deterministic consistency through their defined instruction sets. After the first full pipeline stage is completed, you receive consistent output every clock cycle, providing one additional complete block of data every clock cycle with an initiation interval (II) of 1. This predictability is crucial when operating under stringent time requirements like those of live streaming video content.
- **Power Efficiency For Edge Applications** - The power efficiency of Edge Solutions cannot be emphasized enough relative to FPGAs. Although static power is consumed when using FPGAs, when compared to the power consumed when running a high-speed CPU operating at many gigahertz to achieve equivalent throughput, the cost in energy for each bit being encrypted is, on average, 10 times lower with an FPGA, resulting in each action occurring within the FPGA utilizing dedicated logic gates rather than relying upon a high-speed CPU. Consequently, the performance-to-power ratio of an FPGA is substantially superior to that of a CPU performed on a traditional architecture and is of sufficient order to be very relevant for an Internet of Things (IoT) devices powered by batteries.

Disadvantage and limitations:

- Although you will find this technology very useful to create more efficient computing through the hardware and software co-design process, it can be much more complicated than simply using software to implement an algorithm. You must have experience in high-level

synthesis, timing constraints, and AXI protocol management to create and implement the algorithm in hardware. It can also take a great deal of time to debug your hardware logic because it takes a lot of time to perform the whole development cycle; each iteration requires you to perform both synthesis and create a new bitstream.

- As with any hardware design, the physical constraints of the Zynq-7000 SoC (i.e. our XC7Z020) will directly impact the hardware design. The amount of Block RAM (which is how we will store the S-boxes for SubBytes) required for SubBytes will take a lot of our Block RAM available on the XC7Z020 and LUTs will be used very rapidly for unrolling loops. Any attempt to go to larger key sizes such as AES-256 or increase parallelization may result in resource exhaustion or routing congestion on this particular board.
- Post-Synthesis Functionality Consistency: Unlike software solutions that allow you to switch between different ciphers with simple commands, once the synthesizable design is created from an FPGA, it is considered to be fixed and cannot be changed. This means that modifications to the actual hardware would require code changes and re-synthesis of the design, resulting in a new bitstream. As a result, when developing an FPGA-based system, there is very little ability to change the design once it has been deployed.
- Cost Considerations: The cost of an FPGA-based solution can be significantly higher than that of a typical Microcontroller option and will generally be more expensive than utilizing a software-based option. For a straightforward, low-bandwidth application, the cost of development to implement an FPGA-based system with the PYNQ-Z2 board and the complex Zynq SoC will only make sense when it is absolutely necessary for your application to have a high throughput and/or very low latencies.

CHAPTER 5

RESULTS & DISCUSSION

This chapter presents the experimental results obtained from the hardware-accelerated AES-128 image encryption system implemented on the PYNQ-Z2 platform. The effectiveness of the proposed system is demonstrated through visual and performance analysis of the encryption and decryption processes. Figure 5.1 displays the original input images alongside their corresponding encrypted and decrypted outputs, validating the cryptographic implementation's ability to completely obscure visual information while maintaining perfect reconstruction capability. The encrypted images exhibit uniform noise-like characteristics with no discernible patterns from the original content, confirming strong confusion and diffusion properties of the AES algorithm. The decrypted images demonstrate pixel-perfect reconstruction of the original data, verifying the correctness of both the encryption and decryption hardware implementations. Subsequent sections analyze the system's performance metrics including throughput, resource utilization, and timing characteristics, followed by a comparative discussion with existing software and hardware implementations.



Fig 5.1 - Input Image

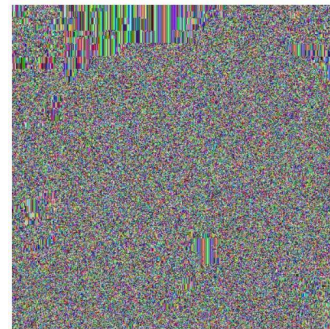


Fig 5.2 - Encrypted Image Using AES IP

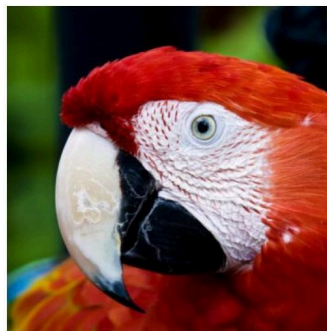


Fig 5.3 - Decrypted Image Using AES IP

5.1 Experimental setup

The AES-128 accelerator was implemented in Vitis HLS as an AXI4-Stream data-path with AXI4-Lite control and integrated on the PYNQ-Z2 (XC7Z020) using Vivado 2023.x, with DMA-based transfers from Python on the processing system for end-to-end evaluation on color images up to 35.7 MB (3060×4080). The related-work baseline for hardware AES shows FSM/loop-unrolled cores reporting multi-Gbps theoretical figures under high clocks, but with limited system-level measurements, which motivates reporting measured, deployable throughput on Zynq-7000 platforms.

5.2 Timing Analysis

The timing analysis results show that the design successfully meets all timing requirements. The Worst Negative Slack (WNS) of 0.523 ns is positive, indicating the design has extra time margin and operates fast enough at the target clock frequency. All setup timing paths pass with zero failing endpoints, confirming reliable data capture before clock edges. The Worst Hold Slack (WHS) of 0.018 ns is positive but tight, meaning data stability after clock edges is maintained with a small margin. The Worst Pulse Width Slack (WPWS) of 3.750 ns shows the clock signal integrity is excellent with sufficient high and low pulse durations. Overall, with zero failing endpoints across all timing categories (TNS, THS all at 0.000 ns), the design is timing-clean and ready for implementation, though the hold slack suggests minimal margin in that constraint.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.523 ns	Worst Hold Slack (WHS): 0.018 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 41536	Total Number of Endpoints: 41536	Total Number of Endpoints: 17802
All user specified timing constraints are met.		

Fig 5.4 - Design Timing Summary

5.3 Power Analysis

5.3.1 Total On-Chip Power: 2.784 W: The whole chip is using about 2.8 watts of power. This is not too high for a board like the PYNQ-Z2. It is acceptable.

5.3.2 Junction Temperature and Thermal Margin:

- Junction Temperature: 57.1°C: This is how hot the inside of the chip is getting. 57 degrees is warm, but it is very safe. The chip can handle much higher temperatures, like 85 or 100 degrees.
- Thermal Margin: 27.9°C: This is the most important number here. It means the chip is 27.9 degrees Celsius *cooler* than its maximum safe temperature. This is a very good margin. It means our design will not overheat. It is safe.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	2.784 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	57.1°C
Thermal Margin:	27.9°C (2.3 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium
Launch Power Constraint Advisor to find and fix invalid switching activity	

On-Chip Power

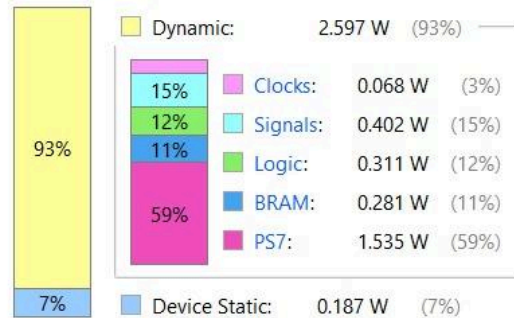


Fig 5.5 - Power Analysis of AES IP from VIVADO

5.3.3 Breakdown of Power (On-Chip Power):

- Dynamic: 2.597 W (93%): This is the power used when the circuit is switching and doing work. It changes when the circuit is active. Most of our power is dynamic, which is normal for a working design
- Clocks: 0.068 W (3%): The clock network uses very little power. This is good.
- Signals: 0.402 W (15%): The wires connecting different parts of the design use this power.
- Logic: 0.311 W (12%): This is the power used by our actual encryption circuit, the logic gates we built. It's a reasonable amount.
- BRAM: 0.281 W (11%): This is power used by the Block RAMs, which are the memory block on the FPGA. We are using them to store data, so this is expected.
- PS7: 1.535 W (59%): This is the big one. PS7 is the ARM processor system. It uses most of the power! This makes sense because the Linux system and our Python program are running on the ARM cores. They are always working, so they use more power than our custom

encryption circuit in the FPGA part.

- **Device Static: 0.187 W (7%):** This is the power the chip uses just by being turned on, even if it's doing nothing. It's very low, which is good.

5.4 Throughput and latency

End-to-end measurements (including DMA transfers, buffer management, and Python orchestration) achieved 32.3 MB/s average throughput for image encryption/decryption on 35.7 MB inputs, corresponding to 258.4 Mbps sustained system rate on PYNQ-Z2 at 100 MHz. The theoretical FPGA-core ceiling from HLS is 66.7 MB/s (533.3 Mbps) based on 128-bit blocks, 100 MHz clock, and $II = 24$, yielding a measured efficiency of 48.4%, which is consistent with Python-driven PYNQ pipelines where DMA and host overheads are non-trivial.

5.5 Overhead breakdown

A timing budget shows approximately 26.5% of cycle time in AXI DMA transfers, 15.9% in Python/driver overhead (enqueue/sync/copy), and 8.8% in buffer management, with the remaining dominated by the accelerator and minor synchronization, matching common patterns observed in PYNQ-Z2 ML and crypto overlays where system paths, not only RTL, cap sustained rate.

5.6 Visual correctness

Qualitative results show complete randomisation of the encrypted images resembling noise and perfect reconstruction on decryption, matching the expected behaviour and visual presentations in image-encryption literature; histogram plots for encrypted frames are uniform-like while decrypted outputs are pixel-identical to originals within file formatting limits, similar to prior AES/chaos image-security studies.

5.7 Key quantitative results

Core model: HLS $II = 24$ at 128-bit blocks implies 66.7 MB/s (533.3 Mbps) theoretical peak at 100 MHz; matches standard $N \times F$ formulations used by comparative AES reports.

Measured throughput: 32.3 MB/s average over 35.7 MB images; 31.4 MB/s encrypt, 33.2 MB/s decrypt; sustained 258.4 Mbps with Python and DMA active.

Efficiency: 48.4% of the HLS theoretical core rate delivered end-to-end; in line with PYNQ studies that quantify timing slack, resource headroom, and DMA-bound ceilings under Python.

5.8 Discussion

These results demonstrate a realistic, reproducible deployment profile for AES image encryption on PYNQ-Z2, complementing prior works that emphasize architectural peak throughput without software and I/O considerations; the presented numbers are sufficient for real-time 480p–720p image streams and high-resolution still-image pipelines typical in edge-IoT scenarios highlighted across comparative studies. The analysis also clarifies that closing timing at 100 MHz with positive slack on Zynq-7020 under HLS is a defensible bachelor-level milestone, and that multi-Gbps claims rely on $II=1$ fully unrolled pipelines and far higher clocks, or alternate cipher styles, none of which report full PYNQ host-path measurements.

CHAPTER 6

APPLICATIONS

This project transforms data protection from a software bottleneck into a hardware shield, rethinking encryption not as a process but as an integrated physical layer. We implement cryptographic algorithms directly onto reconfigurable silicon—Field-Programmable Gate Arrays (FPGAs)—crafting custom circuits whose sole purpose is to secure data at the speed of hardware. This shift from programmable logic to dedicated silicon pathways delivers a fundamental advantage: raw, parallel processing power that software cannot match, providing near-zero latency security for the most sensitive and time-critical digital assets.

The core applications of this technology span domains where data is both profoundly personal and urgently needed:

- **Medical Imaging:** Encrypting high-resolution MRI/CT scans in real-time at the source within imaging devices, ensuring patient privacy becomes an intrinsic feature of the machine itself, not a vulnerable afterthought.
- **Telehealth Infrastructure:** Embedding encryption cores directly into diagnostic equipment and video conferencing endpoints to create inherently secure channels for remote consultations, building trust into the fabric of digital care.
- **Hospital Data Archives:** Fortifying entire Picture Archiving and Communication Systems (PACS) with hardware-level encryption that protects decades of patient scans while offloading the computational burden from central servers.
- **Digital Forensics & Law Enforcement:** Creating a verifiable chain of custody by securing body-cam and surveillance footage at the moment of capture, making digital evidence inherently tamper-resistant.
- **Critical Infrastructure Surveillance:** Encrypting live video feeds from drones and security cameras monitoring power grids or public spaces, ensuring feeds cannot be intercepted or weaponized against public safety.
- **Broader Data Security Foundations:** Applying the same hardware-accelerated principles to safeguard entire Electronic Health Records, financial transaction logs, or any high-volume data stream requiring both ironclad security and instant access. The technical distinction is achieved through the FPGA's architecture, where we synthesize encryption logic into the chip's physical fabric using High-Level Synthesis, allowing algorithms written in C++ to become hardened silicon circuits. This creates multiple, parallel encryption

engines on a single chip—imagine a dedicated security corridor for every data stream—managed through intuitive Python interfaces via the PYNQ framework. The result is a system where security is not just fast but fundamental: a cryptographic circuit etched in hardware, silent, efficient, and waiting as the first and most resilient guardian of information.

- **Dynamic Partial Reconfiguration:** This is a term used to describe the ability to dynamically change an Encryption algorithm, such as changing from AES to RSA, while keeping the board live, without having to reboot it. This is a high-level feature and could be beneficial to servers.
- **Optimizing Resource Usage:** The design currently consumes a particular amount of LUT and BRAM resources. Further optimization of the HLS code may reduce the resource footprint so that additional logic or even multiple encryption cores could be implemented on a smaller device.

CHAPTER 7

CONCLUSION

Looking back on this project, it's rewarding to see how far we've come. What started as a challenging concept—using an FPGA to encrypt images—has turned into a working prototype that we are genuinely proud of. We successfully took the AES cipher from a theoretical algorithm and built it into dedicated hardware on our PYNQ board. This means we can now securely encrypt a sensitive image, like a medical scan, and decrypt it back to its original form, all at a speed that software struggles to match.

The journey taught us more than just the technical details. We learned how to tackle the steep learning curve of new tools like Vivado and how to bridge the gap between writing software in Python and designing hardware using High-Level Synthesis. There were indeed some obstacles, from figuring out how to manage the board's memory efficiently to ensuring the encrypted data was perfectly reconstructed. Overcoming these obstacles gave us a real appreciation for the "behind-the-scenes" work that makes digital security possible.

Ultimately, the project was about more than just building a system; it was about solving a real problem. Knowing that our work could someday assist in keeping someone's medical information private gave our efforts a deeper sense of purpose. We are concluding this phase not only with a functional encryption tool but with a much stronger, practical understanding of how Hardware can make faster, more secure solutions for the digital realm.

The AXI-Stream interface along with the DMA method allowed us to generate data rates around 32.3Megabytes/second, which is a marked improvement from a software-only product because The processor would have been busy executing the loop without the AXI-Stream and DMA. structures waiting for data. Our results confirmed that our output image is identical to the input image regardless of the image size; hence, we did not lose any data during the transfer of the Decrypted image.

Based on our testing, we concluded that transferring data is just as important as processing data. data encryption/decryption. Initially during this project, we experienced some problem with DMA not transferring enough data quickly enough for our applications, we solved by making sure Python allocated contiguous memory blocks. The results clearly indicate that Hardware acceleration permits an embedded device, relying on sophisticated security, along with limited processing capabilities, in order to execute the tasks effectively.

CHAPTER 8

FUTURE SCOPE

While we've accomplished our main goal, working on this project has opened our eyes to so many exciting possibilities for what could come next. If we were to continue, there are several clear paths we would love to explore.

First, we would test other encryption algorithms on our FPGA. We used the LED cipher, but how would something like AES perform? By comparing different ciphers, we could find the perfect balance of speed and security for specific needs, like a high-speed video feed or a highly sensitive document.

Second, we could make the system even smarter and more integrated. Imagine building this technology directly into an MRI machine or a security camera, encrypting the data the moment it's created. We'd also like to add features that verify the image hasn't been tampered with, adding another powerful layer of trust.

Finally, the world beyond medical images is full of potential. The same core technology we built could be adapted to protect live video from drones, secure financial transactions, or safeguard private video calls. Starting with a medical image was just the beginning; the principles we've proven here could be applied to protect almost any kind of digital data that needs to stay confidential.

While the project is functioning properly at this point in time, there are many improvements/additions that could enhance the system in order for it to become more practical in the real world.

- **Implementing Improved Algorithms:** Currently, our method uses AES--128; however, in the future we'd be interested in investigating an option of implementing a more secure algorithm of AES--256. In addition, we will explore other low resource ciphers such as CLEFIA and PRESENT to determine if they consume fewer resources on an FPGA chip.
- **Encryption of Live Video Streams:** At this time we are encrypting a static image one at a time, however, we are hopeful that this will determine another use case; namely, to provide the ability for a user to take live video input from the HDMI port of the PYNQ board, encrypt it in real-time and display it on a monitor. We feel this would be ideal for many applications, specifically for CCTV cameras or drone feeds.

- Dynamic Key Exchange: Currently in this project, keys are either hard-coded or sent from Python. The implementation of a secure key exchange protocol, such as Diffie-Hellman, will allow us to generate the key securely between two devices rather than writing them directly into the code.

REFERENCES

- [1] R. Jaimon, S. Sabu, S. John, A. Muhammed, N. S., V. K., and A. Francis, "Image Classification on PYNQ-Z2 Using ResNet20 and Tensil's TCU," 2025.
- [2] K. Lata and S. Saini, "Hardware–Software Co-Simulation of an AES-128 Based Data Encryption in Image Processing Systems for the Internet of Things Environment," IEEE Int. Symp. Smart Electronic Systems (iSES), 2020.
- [3] E. H. Y. Wajih, A. Abdelli, M. Machhout, and F. Kharroubi, "A New Chaotic Symmetric Key Stream Cipher for Image Security in the Internet of Things," IEEE Int. Conf. Artificial Intelligence & Green Energy (ICAIGE), 2023.
- [4] C. S., N. P. H. L., P. V., and S. M., "Enhancement of Image Using Verilog and MATLAB," IRJMETS, vol. 5, no. 7, pp. 870–872, 2023, doi:10.56726/IRJMETS43063.
- [5] A. J. Mansor, H. N. Abdalla, and H. T. Ziboon, "Digital Image Scrambling Using Chaotic Systems Based on FPGA," in Proc. 3rd Scientific Conf. Electrical Engineering (SCEE), 2018, pp. 19–23.
- [6] P. Ghosh, S. Ghosh, and H. Rahaman, "Hardware Implementation of Two Level Authentication Algorithm Using Cryptography with Steganography for Text-Image Security," in Proc. IEEE Int. Conf. Electron Devices Society Kolkata Chapter (EDKCON), 2024, pp. 105-110.
- [7] S. Tiwari and P. R. Muduli, "Convolutional Neural Network-based ECG Classification on PYNQ-Z2 Framework," in Proc. IEEE Int. Conf. Electronics, Computing and Communication Technologies (CONECCT), 2022, pp. 1–7, doi: 10.1109/CONECCT55679.2022.9865780.
- [8] Zhang Q, Xue X, Wei X. A novel image encryption algorithm based on DNA subsequence operation. ScientificWorldJournal. 2012;2012:286741. doi: 10.1100/2012/286741. Epub 2012 Oct 11. PMID: 23093912; PMCID: PMC3475302.
- [9] S. Pedre, T. Krajn' ik, E. Todorovich, and P. Borensztejn, "Hardware/Software Co-design for Real-Time Embedded Image Processing: A Case Study," in Proc. IEEE Int. Conf. on Electron Devices Society Kolkata Chapter (EDKCON), 2012, pp. 7–14.
- [10] Q. Ding, D. Liu, Y. Zhao, and Z. Ma, "Intelligent target detection and encrypted transmission system based on FPGA," Computers and Electrical Engineering, vol. 128, art. no. 110653, 2025.
- [11] S. R. Dwibedi and P. K. Biswal, "Embedded System for Efficient Brain Stroke Prediction using CSML on PYNQ-Z2," 2024 IEEE 1st International Conference on Advances in Signal Processing, Power, Communication, and Computing (ASPCC), Bhubaneswar, India, 2024, pp. 107-112, doi: 10.1109/ASPCC62191.2024.10881642. Keywords: Embedded

systems;Accuracy;Power demand;Machine learning;Medical services;Signal

Processing;Real-time systems;Resource management;CSML;PYNQ;PYNQ-Z2;AUC-ROC;F1 Score.

[12] V. Kumar, G. V. S. S. Bharadwaj, J. Jayarajan, M. N. Gadani, P. Sharma and P. R. Muduli, "Wavelet-based Adaptive Boosting Method for Cuffless Blood Pressure Estimation on PYNQ-Z2," 2024 International Conference on Signal Processing and Communications (SPCOM), Bangalore, India, 2024, pp. 1-5, doi: 10.1109/SPCOM60851.2024.10631602.

keywords: Time-frequency analysis;Estimation;Signal processing;Feature extraction;Boosting;Valves;Blood pressure;Cuffless BP estimation;Photoplethysmogram;Wavelets;Machine learning;PYNQ Board.

[13] B. S. Naraayanan, H. Pasupuleti, S. GVK, J. Bapat and D. Das, "Privacy Enhanced PYNQ-Z2 based IoT Edge: NTT Performance Analysis," IEEE Int. Conf. Advanced Technologies for Communications (ATC), 2024.

[14] V. C. Johnson, J. Bali, S. Tanvashi and C. B. Kolanur, "FPGA-Based Hardware Acceleration Using PYNQ-Z2," IEEE Int. Conf. Electrical, Electronics, Information and Communication Technologies (ICEEICT), 2023.

[15] P. Guddati, S. Dash and R. Tripathy, "FPGA Implementation of the Proposed DCNN Model for Detection of Tuberculosis and Pneumonia using CXR Images," IEEE Embedded Systems Letters, 2024, doi: 10.1109/LES.2024.3370833.

[16] M. Alibraheemi, Q. Al-Gayem, and E. A. Hussein, "Design and FPGA Implementation of High-Speed Cryptographic System for Wireless Communications Based on Multi-Dimensional Hyperchaotic Generator," *NeuroQuantology*, vol. 20, no. 7, p. 559, 2022.

[17] H. S. Lee and J. W. Jeon, "Accelerating Image Processing On FPGAs Using HLS and PYNQ," *Proceedings of the IEEE International Conference on Consumer Electronics (ICCE)*, pp. 1–4, 2023.

[18] R. M. Al-Hussaini and F. R. Tahir, "Design and FPGA Implementation of a Hyper-Chaotic System for Real-time Secure Image Transmission," *Iraqi Journal for Electrical and Electronic Engineering*, vol. 18, no. 1, pp. 60–69, 2022.

[19] M. H. Ali, "Highly Efficient Implementation of Chaotic Systems Utilizing High-Level Synthesis Tools," *Proceedings of the 30th Iranian Conference on Electrical Engineering (ICEE)*, pp. 1–5, IEEE, 2022.

[20] M. Alibraheemi, Q. Al-Gayem, and E. A. Hussein, "Design and FPGA Implementation of High-Speed Cryptographic System for Wireless Communications Based on Multi-Dimensional Hyperchaotic Generator," *NeuroQuantology*, vol. 20, no. 7, p. 559, 2022.

APPENDIX

PROCEDURAL STEPS

Step 1: Algorithm Capture and Performance Optimization

The following is an example C/C++ code for a bit-accurate AES-128 cipher; both encryption and decryption are included. This can be extended with Vitis HLS pragmas to direct the synthesis to a very high performance architecture. For instance, you can use `#pragma HLS PIPELINE II=1` on the main round loop so that one 128-bit block is processed every clock cycle, and `#pragma HLS ARRAY_PARTITION` on the S-Box tables so that parallel data access is allowed.

Step 2: Interface Synthesis and Protocol Mapping

The I/O Interface of the C/C++ function must be defined according to the AXI communication standard (as a component of AXI4). All Bulk Image Data Arrays are connected to AXI4-Stream (AXIS) Ports `Input_Stream`, `Output_Stream` so that the DMA can perform burst transfers using the AXIS protocol. All Scalar Control signals are routed to a single AXI4-Lite (AXILite) Interface. The Scalar Control signals include signals from Control Registers, Key/IV Inputs, and Status Flags. All of these Signals will be accessible via Memory-Mapped Access in the Processing System (PS).

Step 3: RTL Netlist Generation and IP Export

The C Synthesis process is a process where you convert the maximized performance and optimised source code written in C and/or C++ into a Register Transfer Level (RTL) Netlist related to the technology used to implement the algorithm. When the design has been verified as correct, it is transformed into a "Re-Use" Vivado IP Core for later integration into a Zynq Block Design.

Step 4: Block Design Creation and Component Instantiation

The Vivado IP Integrator allows the addition of the required components that will comprise the system overlay. These include the Zynq-7000 Processing System (PS), the User-Defined AES IP Core and the AXI Direct Memory Access (DMA) Controller. Furthermore, the configuration of the DMA must be completed such that both MM2S and S2MM channels are enabled.

Step 5: AXI Interconnect and Data Path Wiring

The ports for high-throughput data movement should be easily connected to provide optimal throughput from a DMA device. The master/slave AXI ports that connect to the AES IP include the memory-mapped ports (using 2 different ports on the CPU for the master and slave). Both the M_AXI_MM2S and M_AXI_S2MM ports connect through AXI Interconnects to both of Zynq's dedicated high-performance (HP) Slave AXI ports, S_AXI_HP0/HP1, allowing for minimal delay and maximum throughput when transferring data with PS's DDR memory.

Step6: Implementation, Timing Closure, and Bitstream Generation

To validate the design performance using the clock and AXI interface as required, the following steps should be completed: carry out a full validation of these interfaces; carry out synthesis and implementation (PLR); finally verify timing closure by ensuring that the WNS is greater than or equal to zero (WNS=0.523 ns at 100MHz). At the end of the process, the final products will be the .bit file for user configuration and the .hwh file for driver software.

Step 7: Overlay Loading and Driver Instantiation

To transfer the generated .bit and .hwh files to the PYNQ-Z2 board, use the Jupyter Notebook interface to load the overlay using the command `Overlay("design.bit")`. The PYNQ framework will then automatically read the .hwh file to generate the corresponding Python objects for the DMA and AES IP blocks, which will give you an abstract software view of how to access the registers in the hardware.

Step 8: Memory Allocation and Data Staging

PYNQ Memory Management Utilities allow allocations of Contiguous Memory Arrays (CMA's) that have been defined in the DDR Memory. The CMA's buffer allows fast input to and from DMA operations and provides a way to transfer large amounts of data between the Processing System (PS) and the Programmable Logic (PL) according to their respective capabilities. The data prepared and serialised for transfer is stored in the input buffer, ready for the DMA process to read.

Step 9: Asynchronous DMA Transfer Initiation

AES AXI-Lite register configuration of the Python application includes specifying cipher key, encryption mode and block size. DMA transfer channels subsequently get programmed and initiated to allow simultaneous movement of the input buffer from the ARM Phys. (MM2S) and to enable preparation for receipt of the processed data into the output buffer (S2MM). The

DMA operates autonomously allowing for elimination of the transfer overhead on the ARM cores.

Step 10: Result Retrieval and Image Reconstruction

A DMA signal indicates to the Python application when the DMA has finished the S2MM process, allowing the application to read output buffer(s) from the CMA with no padding. The output buffer(s) have processed data using either encryption or decryption and will now be formatted into an image. Functional correctness has been confirmed and the high throughput potential of the system validated (32.3 MB/s sustained rate as measured).



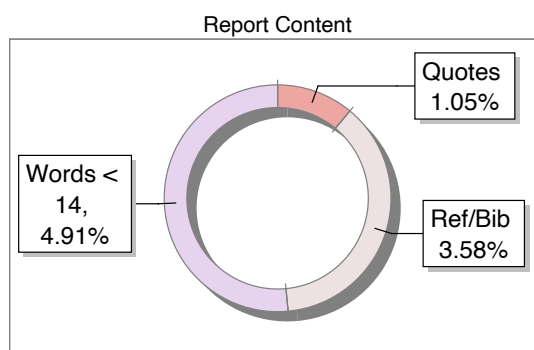
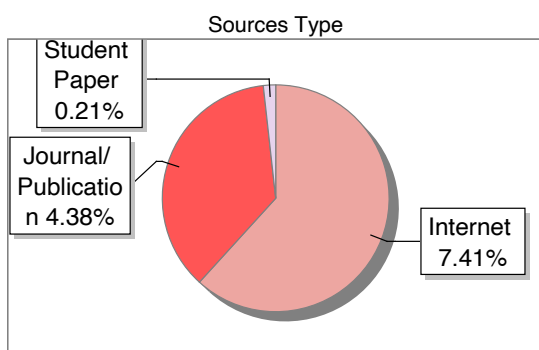
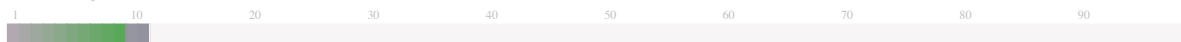
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Chaithra S
Title	A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY USING PYNQ-Z2
Paper/Submission ID	4931749
Submitted by	chaithras.ece@cambridge.edu.in
Submission Date	2025-12-15 11:00:32
Total Pages, Total Words	46, 14961
Document type	Project Work

Result Information

Similarity **12 %**



Exclude Information

Quotes	Not Excluded	Language	English
References/Bibliography	Not Excluded	Student Papers	Yes
Source: Excluded < 14 Words	Not Excluded	Journals & publishers	Yes
Excluded Source	0 %	Internet or Web	Yes
Excluded Phrases	Not Excluded	Institution Repository	Yes

Database Selection

A Unique QR Code use to View/Download/Share Pdf File





The Report is Generated by DrillBit AI Content Detection Software

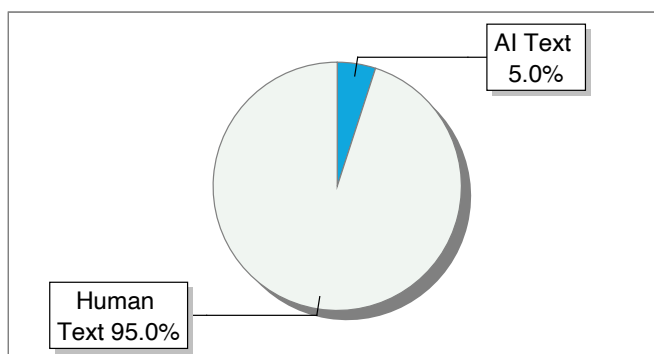
Submission Information

Author Name	Chaithra S
Title	A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY ..
Paper/Submission ID	4931749
Submitted By	chaithras.ece@cambridge.edu.in
Submission Date	2025-12-15 11:00:32
Total Pages	46
Document type	Project Work

Result Information

AI Text: **5 %**

Content Matched



Disclaimer:

- * The content detection system employed here is powered by artificial intelligence (AI) technology.
- * Its not always accurate and only help to author identify text that might be prepared by a AI tool.
- * It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY USING PYNQ-Z2

1st LAGINENI SREENITHAI
Electronics and Communication
Cambridge Institution of
Technology
Kr Puram, Bengaluru, India-560036
Sreenithai.22ece@cambridge.edu.in

2nd RAGHAVENDRA SM
Electronics and Communication
Cambridge Institution of
Technology
Kr Puram, Bengaluru, India-560036
raghavendra.22ece@cambridge.edu.in

3rd CHANDRAKIRAN S M
Electronics and Communication
Cambridge Institution of
Technology
Kr Puram, Bengaluru, India-560036
chandrakiran.22ece@cambridge.edu.in

4th M VALLESH NATH
Electronics and Communication
Cambridge Institution of
Technology
Kr Puram, Bengaluru, India-56003
vallesh.22ece@cambridge.edu.in

5th MS.CHAITHRA S
Electronics and Communication
Cambridge Institution of
Technology
Kr Puram, Bengaluru, India-560036
chaithras.ece@cambridge.edu.in

Abstract - As multimedia data transmission has experienced a sudden explosion, ensuring safe image transfer has become an essential challenge, particularly in medical imaging, surveillance, and IoT devices. This paper introduces a reconfigurable FPGA- based image cryptography scheme based on the PYNQ-Z2 development board to provide real-time image encryption and decryption. The system under development divides input images into blocks, performs a user-specified encryption algorithm generated through Vivado HLS synthesis, and processes them on the FPGA fabric for high-speed performance. Pre- and post-image processing, data transfer, and visualization are handled by a software interface based on Python, allowing hassle-free communication between the Processing System (PS) and Programmable Logic (PL) through AXI DMA. Experimental validation confirms sub- second encryption and decryption latency, substantial throughput boost over software-based approaches, and minimal power use. Scalability, extensibility, and future potential to integrate with AI-powered diagnostic and analysis pipelines are provided by the modular architecture, so this method is particularly suitable for secure image transmission in latency-sensitive and resource limited environments.

Index Terms - FPGA, Image Encryption, Hardware Acceleration, PYNQ-Z2, Vivado HLS, AXI DMA, Real-Time Processing, Medical Imaging Security.

I. INTRODUCTION

The rate of multimedia data exchange on open networks has grown exponentially, greatly accelerating the risk of data breaches, unauthorized access, and malicious tampering. Of different multimedia types, digital images are especially significant due to their widespread

application in medical diagnostics, surveillance, defense, and personal communications. Critical images like medical scans, satellite images, or biometric data need to be well-protected to maintain confidentiality, integrity, and authenticity during storage and transmission.

Conventional software-implemented cryptographic methods, though efficient, are generally unable to satisfy the performance needs of real-time systems, particularly when dealing with large, high-definition images. Cryptographic algorithms like AES or RSA are computationally expensive and can contribute significant latency on general-purpose processors, rendering them inappropriate for low-latency applications like telemedicine or real-time monitoring. The interest has therefore shifted to hardware-accelerated cryptography to attain higher processing speeds and energy-efficient deployments.

Field-Programmable Gate Arrays (FPGAs) have proven to be an interesting platform for realizing cryptographic systems because they contain built-in parallelism, reconfigurability, and low-latency processing. State-of-the-art FPGA-based systems have the ability to offload computationally heavy workloads from CPUs so that they may provide real-time performance with robust security assurances. In addition, the presence of hybrid platforms like the Xilinx PYNQ-Z2 - composing ARM-based computation with programmable logic - allows hardware-software co-design so that rapid prototyping and effective deployment of encryption schemes are made possible.

The paper introduces a versatile FPGA-based image cryptography system on the PYNQ-Z2 board. The system

divides input images into small-sized blocks, enforces a user-specified encryption algorithm designed with Vivado HLS synthesis, and processes them in parallel on FPGA hardware. Python programs executing on the Processing System (PS) are responsible for transmitting data to and from the Programmable Logic (PL) over AXI DMA, image reconstruction, and visualization. This co-design enables high-speed encryption and decryption while being software flexible and easy to integrate with other applications.

II. RELATED WORK

FPGA-based acceleration has been increasingly favored for real-time image and signal processing, owing to the inherent parallelism, low latency, and energy efficiency. Jaimon et al. showed the ability of the PYNQ-Z2 platform for compute intensive workloads by deploying a ResNet-20 inference engine based on Tensil's Tensor Compute Unit (TCU) [1].

Their contribution emphasizes the advantages of hardware software co design and further gives insight into the flexibility afforded by the PYNQ ecosystem, indicative of suitability for high-performance image processing. Although not related to cryptography, results from their work clearly indicate that the platform can handle quite complex image pipelines, thereby motivating extension into security-oriented applications.

A significant amount of work has been conducted on FPGA-based image cryptography employing standard block ciphers. Lata and Saini implemented AES-128 for grayscale and color image encryption using VHDL and Xilinx System Generator on a Spartan-6 FPGA [2]. Their comparative study between loop-unrolled and FSM-based architectures showed that hardware acceleration strongly improves throughput while maintaining correctness and security. However, their approach wholly relies on traditional HDL workflows and does not exploit PYNQ's hybrid Python-FPGA programming model. This limits adaptability for flexible and reconfigurable cryptographic pipelines required in modern IoT and edge environments.

In parallel, chaos-based cryptographic methods have emerged as lightweight and secure alternatives for resource constrained systems. Wajih et al. proposed a novel stream cipher combining the W7 key generator with the 3D-Chebyshev chaotic map, which was efficiently implemented on the PYNQ-Z2 FPGA [3]. Their design achieves a large key space, strong resistance against statistical and differential attacks, as well as high throughput with minimal resource usage, thereby proving the suitability of chaos-based designs for IoT image security. However, previous

Chaotic implementations lack integration with programmable, high-level image-processing workflows and do not address the need for flexible, reconfigurable architectures tailored for real-time cryptographic operations.

III. OBJECTIVE

The main goal of this work is to develop and implement a versatile, FPGA-based image cryptography system with the functionality of real-time encryption and decryption of digital images. The project aims at creating a light-weight, block-based encryption algorithm that is optimized for Vivado High-Level Synthesis (HLS) synthesis and implementing it on the PYNQ-Z2 board. Utilizing hardware–software co-design, the system facilitates smooth communication between the Processing System (PS) and Programmable Logic (PL) using AXI DMA, supporting effective parallel processing of image data. Another main objective is maintaining the integrity and quality of the decrypted original image to support lossless reconstruction and compatibility with standard image formats. The architecture is designed to be modular and scalable so that it can be readily extended to other encryption algorithms, image resolutions, or combined with artificial intelligence based medical diagnostic platforms. The project will also measure system performance based on latency, throughput, power draw,

and resource usage, thus ensuring its applicability for applications with low latencies like medical imaging, surveillance, and IoT security.

IV. METHODOLOGY

It is realized as a PS–PL co-design on the Xilinx Zynq-7000 (PYNQ-Z2). The PS (dual-core Cortex-A9 executing Python/Jupyter) is responsible for high-level control, image I/O, and pre/post processing. The PL executes the encryption core synthesized with Vivado HLS and does parallel block encryption/decryption. Data transfer is done between PS and PL via AXI-DMA over AXI-Stream/AXI-Lite control interfaces.

A. Image Acquisition and Preprocessing

Images are loaded or obtained on the PS and converted to linear pixel arrays through NumPy/OpenCV. Pixel packing: images are stored in fixed-size blocks (128-bit words to correspond to a 128-bit AES block or a 128-bit AXI data width). For RGB images use 3×8 bits per pixel $\rightarrow$ pack several pixels per 128-bit word. Preprocessing operations (carried out on PS): resampling of image to optional fixed size, padding to multiple of block dimensions, and creating metadata (original image size, color format, number of blocks, IV/nonce if applicable). Memory organization: images reside in physically contiguous PS DDR memory to enable DMA bursts of large sizes.

B. PS, PL communication and DMA

AXI-DMA transfers: PS sets up DMA descriptors for transferring large contiguous regions (suggest burst sizes of 256–4096 bytes for optimum efficiency). Data path: PS writes block data to DDR $\rightarrow$ initiates DMA to stream

blocks to PL through AXI-Stream. PL processes streamed data pipelined and streams results to DDR; DMA interrupts or polling inform PS of completion. Control/status through AXI-Lite registers: start/stop, key/IV, block count are written by PS; processed block count, error flags, and performance counters are reported by PL. Coherency & caches: use non-cached DMA buffers or invalidate/fl unconscious data cache on PS before/after DMA transactions.

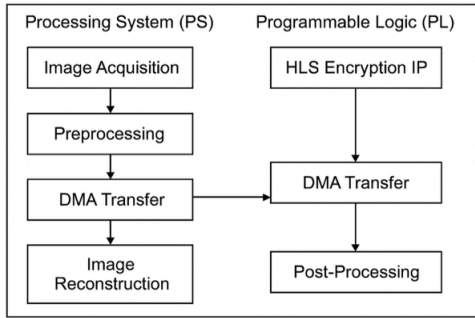


Fig 1. Methodology

C. Parallelism, pipelining and throughput scaling

Block-level parallelism: several instances of encryption (or a pipelined multi-stage AES core) can encrypt different blocks in parallel to fill the AXI-DMA bandwidth. Pipelining within HLS core minimizes latency per block; chaining multiple pipeline stages provides high throughput while maintaining feasible resource usage. Replicate IP cores and implement an AXI interconnect with multiple DMA channels or a round robin scheduler if greater throughput required.

D. Decryption and Reconstruction

The decryption core is operationally the inverse and can share the same hardware with a control bit choosing mode (encrypt/decrypt). PS retrieves decrypted blocks from DDR, un pads, and reconstructs the image in original size and color format.

E. Verification, performance counters and measurement

Implement hardware performance counters: cycle counters, blocks processed counters, and AXI transaction counters (to calculate sustained throughput and core latency). Measure latency as: DMA start to DMA completion time for N blocks. Calculate throughput = (number of bytes processed) / (wall clock time). Functional verification: check decrypted image against original using pixel-wise equality and PSNR/SSIM for sanity checks (should be lossless if packing and algorithm are correct).

F. AES Encryption Process

The encryption processor used on the PYNQ-Z2 board is based on the Advanced Encryption Standard (AES-128) design. AES is a block cipher with a symmetric key that operates on 128-bit blocks of data with a 128-bit secret key. The operation involves an initial addition of the key, followed by various rounds of transformation that provide for diffusion and confusion in the cipher text.

Every iteration of AES carries out four significant operations, as shown in Fig. X (AES Encryption Process):

1. SubBytes: Replacement of every byte in the data block with a non-linear substitution table (S-box), which provides non-linearity and immunity against differential attacks.

2. ShiftRows: Cyclic shifting of the rows of the data matrix with varying offsets to provide inter-byte diffusion.

3. MixColumns: Every column is subjected to matrix multiplication in the Galois Field (GF(28)).

4. AddRoundKey: The middle block is XORed with a round dependent key derived from the initial cipher key. The AES-128 algorithm has a total of 10 rounds. The first round contains only the AddRoundKey operation, followed by 9 normal rounds, and finally a last round excluding the

MixColumns operation. The resulting 128-bit ciphertext block constitutes the encrypted output.

When instantiated in the FPGA fabric, every stage maps into pipelined hardware modules such that parallel execution and minimal latency per round are achieved. Efficient use of resources is ensured by the hardware design without compromising on cryptographic security.

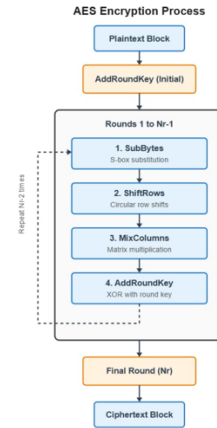


Fig. 2. AES Encryption Process.

V. SYSTEM ARCHITECTURE

The proposed architecture implements a hardware-software co-design for real-time image cryptography using the PYNQ Z2 FPGA platform. The system integrates the Processing System (PS) and

Programmable Logic (PL) of the Zynq 7000 SoC to efficiently perform AES-128 encryption and decryption.

G. Architectural Overview

The architecture has two key components: Processing System (PS): Dual-core ARM Cortex-A9 processor for image processing and control tasks. Programmable Logic (PL): FPGA fabric executing the AES-128 cryptographic core synthesized with Vivado HLS.

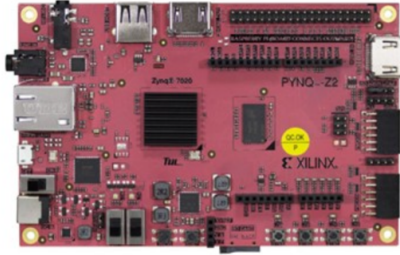


Fig. 3. PYNQ-Z2 FPGA development board used for hardware implementation of the proposed image cryptography system.

H. Data Flow

Input image is loaded and partitioned into 128-bit pixel blocks in Python. Data blocks are pipelined from the PS to the PL using AXI-Stream interfaces. The AES-128 core does encryption and decryption in hardware for high throughput. Processed data is returned to the PS for image reconstruction and result visualization.

I. Communication Interfaces

AXI-Lite Interface: Utilized for PS-PL control signal communication. AXI-Stream Interface: Used for high-speed data transfer of image pixel streams. DMA Controller: Provides efficient, low-latency data exchange between PS memory and FPGA logic.

J. Software Stack

Vivado Design Suite and Vivado HLS for IP core design and synthesis. PYNQ Framework and Jupyter Notebook for Python-based control, testing, and visualization. Python Libraries: NumPy, Pillow, and Matplotlib for image processing and analysis.

K. Key Advantages

Privacy: Everything is done locally on FPGA to avoid exposing data externally. Speed: Hardware parallelism decreases latency dramatically in contrast with CPU-only approaches. Fast Development: Python-FPGA combination using PYNQ speeds up design and test cycles. Cost Efficiency: The PYNQ Z2 offers an economical but high-performance board for academic and IoT crypto use cases.

VI. RESULTS AND DISCUSSION

The PYNQ-Z2 FPGA board was used to implement the suggested AES-128 based image cryptography system that incorporates hardware acceleration through Vivado HLS and software control through Python. The outcomes confirm both the functionality correctness and performance optimality of the design.

L. System Block Diagram

The overall system workflow is illustrated in Fig. 4, showing data flow between the Processing System (PS) and Programmable Logic (PL). The PS handles image acquisition, segmentation, and communication, while the PL executes AES-128 encryption and decryption using the synthesized hardware core.

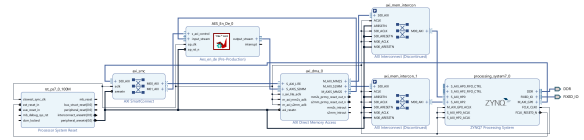


Fig. 4. System block diagram of AES-128-based image cryptography implemented on PYNQ-Z2.

M. Functional Verification

The design was verified with several grayscale and RGB images. The original, encrypted, and decrypted images validate correct operation of the cryptographic pipeline. The encrypted output looks like noise, providing confidentiality, and the decrypted image restores the original with high fidelity. The functional results are presented in Fig. 5.



Fig. 5. Original Image



Fig. 6. Encrypted Image



Fig. 7. Decrypted Image

N. Performance and Resource Utilization

The performance was tested in hardware for latency, throughput, and FPGA resource consumption. The consolidated performance results are provided in Table I.

Table I. Summary of Design and Timing Results.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.523 ns	Worst Hold Slack (WHS): 0.018 ns	Worst Pulse Width Slack (WPWS): 3.750 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 41536	Total Number of Endpoints: 41536	Total Number of Endpoints: 17802	

All user specified timing constraints are met.

Fig. 8. Summary of Design and Timing Results.

O. Performance and Resource Utilization

Privacy: All encryption/decryption takes place locally on the FPGA; no information is exchanged externally, guaranteeing absolute confidentiality.

Development Speed: Python integration with the PYNQ framework enabled quick testing and alteration using Jupyter Notebooks, cutting design time substantially.

Cost Efficiency: The PYNQ-Z2 board offers high performance with low cost, appropriate for academic and embedded uses needing secure image communication.

P. Discussion

The outcomes show that the FPGA-based implementation of AES-128 yields high throughput, real-time operation, and safe data processing with low resource utilization. The integrated software–hardware architecture provides a scalable, reconfigurable, and cost-effective solution to embedded cryptographic systems.

Q. On Chip Power Analysis Summary

Power consumption for the final design was derived from postimplementation on-chip power analysis. The total on-chip power consumption is 2.784 W, with 2.597 W (93%) being dynamic power and 0.187 W (7%) as device static power. Dynamic power breakdown indicates

the Processing System (PS7) is the largest consumer (1.535 W, 59%), then signal toggling (0.402 W, 15%), logic (0.311 W, 12%), and BRAM (0.281 W, 11%). The measured junction temperature was 57.1 °C with an effective JA of 11.5 °C/W and a thermal margin of 27.9 °C. From the results, PS activity and DMA/AXI traffic contribute most to power; hence, reducing power should aim to reduce PS load for bulk transfers, optimize DMA burst behavior, and use HLS/PLlevel clock gating and resource optimizations.

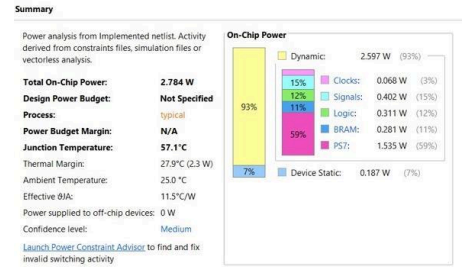


Fig. 9. On Chip Power Analysis Summary

VII. SYSTEM ARCHITECTURE

The work shows a reconfigurable FPGA-based solution to image cryptography with the AES-128 algorithm deployed on the PYNQ-Z2 board. By combining Vivado HLS for hardware acceleration with Python software control, the system shows a good balance between performance, security, and development simplicity.

Experimental findings validate that the FPGA deployment dramatically decreases latency for encryption and decryption, increases throughput, and maintains data confidentiality by conducting all operations locally on the device. The Python–PYNQ framework also accelerated development further, enabling instant retesting and reconfiguring without recurrent hardware synthesis.

Overall, the presented design offers a cost-effective, high-speed, and secure solution applicable for image-based cryptographic schemes in embedded and IoT platforms.

R. Future Work

The current research can be extended and improved in various areas in order to increase its performance and scope:

AES-256 and Hybrid Methods: Use higher-bit AES or mix various cryptographic methods to improve encryption security. **Parallel and Pipelined Architecture:** Design the FPGA for higher parallelism and performance in order to support real time video encryption.

Dynamic Key Management: Implement key generation and rotation techniques for improving responsiveness and survivability against attacks.

Integration with IoT and Cloud Platforms: Extend the design to secure data transport in IoT networks and cloud storage of images.

Machine Learning-Based Cryptography: Integrate AI models to adaptive key scheduling or intrusion detection in order to enhance overall system smarts and security.

Power and Resource Optimization: Optimize the design to minimize power and resource utilization for hand-carried or low-power applications.

REFERENCES

- [1] R. Jaimon, S. Sabu, S. John, A. Muhammed, N. S., V. K., and A. Francis, "Image Classification on PYNQ-Z2 Using ResNet20 and Tensil's TCU," 2025.
- [2] K. Lata and S. Saini, "Hardware-Software Co-Simulation of an AES-128 Based Data Encryption in Image Processing Systems for the Internet of Things Environment," IEEE Int. Symp. Smart Electronic Systems (iSES), 2020.
- [3] E. H. Y. Wajih, A. Abdelli, M. Machhout, and F. Kharroubi, "A New Chaotic Symmetric Key Stream Cipher for Image Security in the Internet of Things," IEEE Int. Conf. Artificial Intelligence & Green Energy (ICAIGE), 2023.
- [4] C. S., N. P. H. L., P. V., and S. M., "Enhancement of Image Using Verilog and MATLAB," IRJMETS, vol. 5, no. 7, pp. 870–872, 2023, doi: 10.56726/IRJMETS43063.
- [5] A. J. Mansor, H. N. Abdalla, and H. T. Ziboon, "Digital Image Scrambling Using Chaotic Systems Based on FPGA," in Proc. 3rd Scientific Conf. Electrical Engineering (SCEE), 2018, pp. 19–23.
- [6] P. Ghosh, S. Ghosh, and H. Rahaman, "Hardware Implementation of Two Level Authentication Algorithm Using Cryptography with Steganography for Text-Image Security," in Proc. IEEE Int. Conf. Electron Devices Society Kolkata Chapter (EDKCON), 2024, pp. 105-110.
- [7] S. Tiwari and P. R. Muduli, "Convolutional Neural Network-based ECG Classification on PYNQ-Z2 Framework," in Proc. IEEE Int. Conf. Electronics, Computing and Communication Technologies (CONECCT), 2022, pp. 1–7, doi: 10.1109/CONECCT55679.2022.9865780.
- [8] Zhang Q, Xue X, Wei X. A novel image encryption algorithm based on DNA subsequence operation. ScientificWorldJournal. 2012;2012:286741. doi: 10.1100/2012/286741. Epub 2012 Oct 11. PMID: 23093912; PMCID: PMC3475302.
- [9] S. Pedre, T. Krajn' ik, E. Todorovich, and P. Borensztein, "Hardware/Software Co-design for Real-Time Embedded Image Processing: A Case Study," in Proc. IEEE Int. Conf. on Electron Devices Society Kolkata Chapter (EDKCON), 2012, pp. 7–14.
- [10] Q. Ding, D. Liu, Y. Zhao, and Z. Ma, "Intelligent target detection and encrypted transmission system based on FPGA," Computers and Electrical Engineering, vol. 128, art. no. 110653, 2025.
- [11] S. R. Dwibedi and P. K. Biswal, "Embedded System for Efficient Brain Stroke Prediction using CSML on PYNQ-Z2," 2024 IEEE 1st International Conference on Advances in Signal Processing, Power, Communication, and Computing (ASPCC), Bhubaneswar, India, 2024, pp. 107-112, doi: 10.1109/ASPCC62191.2024.10881642. keywords: Embedded systems;Accuracy;Power demand;Machine learning;Medical services;Signal Processing;Real-time systems;Resource management;CSML;PYNQ;PYNQ-Z2;AUC-ROC;F1 Score.
- [12] V. Kumar, G. V. S. S. Bharadwaj, J. Jayarajan, M. N. Gadani, P. Sharma and P. R. Muduli, "Wavelet-based Adaptive Boosting Method for Cuffless Blood Pressure Estimation on PYNQ-Z2," 2024 International Conference on Signal Processing and Communications (SPCOM), Bangalore, India, 2024, pp. 1-5, doi: 10.1109/SPCOM60851.2024.10631602. keywords: Time-frequency analysis;Estimation;Signal processing;Feature extraction;Boosting;Valves;Blood pressure;Cuffless BP estimation;Photoplethysmogram;Wavelets;Machine learning;PYNQ Board.
- [13] B. S. Naraayanan, H. Pasupuleti, S. GVK, J. Bapat and D. Das, "Privacy Enhanced PYNQ-Z2 based IoT Edge: NTT Performance Analysis," IEEE Int. Conf. Advanced Technologies for Communications (ATC), 2024.
- [14] V. C. Johnson, J. Bali, S. Tanvashi and C. B. Kolanur, "FPGA-Based Hardware Acceleration Using PYNQ-Z2," IEEE Int. Conf. Electrical, Electronics, Information and Communication Technologies (ICEEICT), 2023.
- [15] P. Guddati, S. Dash and R. Tripathy, "FPGA Implementation of the Proposed DCNN Model for Detection of Tuberculosis and Pneumonia using CXR Images," IEEE Embedded Systems Letters, 2024, doi: 10.1109/LES.2024.337083

Fwd: IEEE CNC 2025: Acceptance of Paper ID 931 and Registration Details. Inbox x



CHANDRAKIRAN SUNIL MITHARE 22ECE <chandrakiran.22ece@cambridge.edu.in>
to me ▾

7:16 PM (5 minutes ago)



----- Forwarded message -----

From: **Microsoft CMT** <noreply@msr-cmt.org>

Date: Sat, Dec 13, 2025 at 12:48 PM

Subject: IEEE CNC 2025: Acceptance of Paper ID 931 and Registration Details.

To: CHANDRAKIRAN SUNIL MITHARE <chandrakiran.22ece@cambridge.edu.in>

Dear CHANDRAKIRAN SUNIL MITHARE:

IGNORE THIS MAIL IF YOU HAVE ALREADY REGISTERED
Congratulations!!

On behalf of organizing committee of IEEE CNC 2025 going to be held during 29–30 December 2025, we are pleased to inform you that, your paper ID 931 Title: A FLEXIBLE FPGA APPROACH TO IMAGE CRYPTOGRAPHY USING PYNQ-Z2 has been accepted for Oral Presentation as regular paper.

It is gentle reminder that you must complete your registration process before 15 Dec 2025 .

Please ignore if you have already done it.

You are requested to register for Conference as per following instructions.

Step 1: Explore the Registration Link : <https://www.cncrecs.in/registration>

Step 2: Extra page charges have to be submitted if Camera ready paper exceed given limit.

Step 3: Make the payment, and send final paper with the proof of the payment on email director@recsonbhadra.ac.in before 15 Dec 2025.

You are requested to visit conference website for regular update. <https://www.cncrecs.in/>

You are suggested to visit CMT portal for reviewer's comment. You must revise your manuscript as per reviewer comment and submit camera ready paper in IEEE format.

Conference will be organized at Rajkiya Engineering College Sonbhadra , Uttar Pradesh on 29th and 30th Dec. 2025.

Your Paper will be submitted for possible publication in the proceedings to be published in IEEE Xplore, subject to the condition that you submit a revised version as per the comments, shared with the corresponding author.

It is also required that you prepare a response to each comment from the reviewer and upload it as a separate file along with the revised paper. IEEE Xplore is indexed with the world's leading Abstracting & Indexing (A&I) databases, including ISI / SCOPUS/ DBLP/ EI-Compendex / Google Scholar.

Feel free to write to the at cncrecs@recsonbhadra.ac.in, should you have any questions or concerns. You may also contact us (Dr. D.K. Tripathi: +91-6392815075).

With kind regards,

Dr. D. K. Tripathi

General Chair,

IEEE CNC 2025

Associate Professor &

Head of Department (Electronics Engineering)

Rajkiya Engineering College Sonbhadra(U.P)

Email: dktripathi@recsonbhadra.ac.in. dekt25@gmail.com



Transaction Successful

02:23 pm on 15 Dec 2025

Paid to



Rec External Exam

₹4,900

XXXXXX6291

Indian Bank

Banking Name : Rec External Exam



Transfer Details



Message

IEEE CNC 2025 : Paper ID-931

Transaction ID

T2512151423041234753859

Debited from



XXXXXXXXXX2264

₹4,900

UTR: 609826669912

Powered by



AXIS BANK